

Sztuczna Inteligencja

w pracy Data Scientist

– Koncepcje, Narzędzia i Metodologie

Szkolenie

- Plan szkolenia i cele
- Aktualne obowiązki
- Pytania, dyskusje, potrzeby
- Elastyczny program szkoleniowy

Mateusz Kulesza

- Solution Architect
- Senior Software Developer
- Consultant and Trainer

Uczestnik

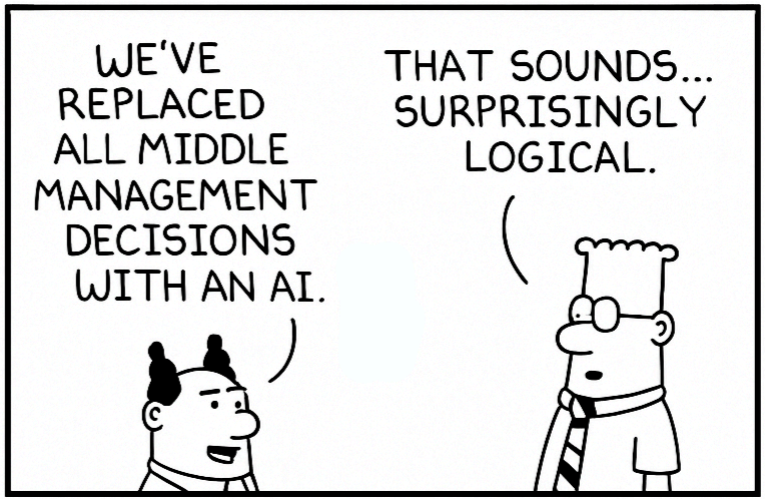
czyli "Ty"

Kilka szybkich pytań:

- Twoje codzienne obowiązki?
- Technologie i background?
- Poprzednie doświadczenie?
- Twoje cele na to szkolenie?
- Określone obszary lub tematy?

Rzeczywistość

w czasach AI



Comic generated by ChatGPT in the style of Scott Adams.

Jak Data Scientist powinien

odróżniać marketing AI od twardej metodologii?

🚀 Hype

– vs. Produktywna Rzeczywistość

🧐 **Dlaczego w ogóle jest hype?**

- **LLM-y wyglądają jak „inteligencja”** → płynny język ≠ głęboka wiedza
- **Marketing producentów modeli**: „agent”, „rozumowanie”, „samodzielność”
- **Media i społeczność** wzmacniają narrację „AI zastąpi Data Scientistów”
- **Większość ocen jest na wyczucie, nie na metrykach**

  – screenshoty odpowiedzi zamiast ewaluacji na zbiorze testowym

🎯 Produktywna rzeczywistość

– co faktycznie działa dziś?

AI = komponent w systemie, nie mózg systemu

- Model jest **funkcją probabilistyczną**, a nie deterministyczną
- Wymaga otoczenia "prawdziwego" systemu:
  - walidacja danych wejściowych i wyjściowych
  - retry + fallback
  - RAG + filtracja danych
  - monitoring driftu i jakości (SLI/SLO)
  - governance
  - i wiele innych ...

Co działa 100% przewidywalnie?

– większość AI nie jest przewidywalne!

Aby mieć gwarancje wyników:

- **Kod EDA i skrypty generowane pod nadzorem** → oszczędza 30–70% czasu
- **AI jako parser / konwerter / etykieciarz** danych
- **Embeddingi + wektorowe wyszukiwanie**
- **Pipeline'y złożone z wielu prostych modeli**, np.
  - OCR → extraction → LLM → decision engine
- **Generowanie dokumentacji, wykresów i testów jakości danych**
- **Asystenci w IDE (Cursor, Copilot++)**

AI jako probabilistyczna część deterministycznego systemu

Co działa, ale z ograniczeniami?

– AI Bezpośredio, ale pod kontrolą

- **Autonomous Agents** → w praktyce 1–3 zadania, nie łańcuchy 100 kroków
- **Fine-tuning** → sens only w przypadku dużych wolumenów i stabilnej domeny
- **AI in the loop** → AI jako element procesu
- **Human in the loop** → AI jako "asystent", człowiek weryfikuje
- **Retrieval Augmented Generation**, ale tylko z:
  - chunkingiem dopasowanym do schematu danych
  - filtrami domenowymi
  - guardrails
  - testami regresyjnymi promptów
  - itd...

Co **NIE DZIAŁA** tak, jak mówi hype

Marketingowa "ściema" i **science-fiction**:

- „AI zrozumie kontekst firmy i będzie jak pracownik”
- „Agent sam ogarnie wieloetapowe zadanie bez kontroli”
- „LLM zastąpi Data Scientista”
- „Model od razu da wiarygodne predykcje bez walidacji”
- „Dzięki AutoML nie trzeba rozumieć danych”
- „RAG rozwiąże wszystkie problemy z wiedzą firmy”

AI nie "myśli", tylko **przewiduje** co będzie dalej:

- Model jest trenowany na opiniach co jest "dobrze" a co "źle"
- Im lepsze dostaje dane wejściowe tym lepsze predykcje
- Im lepiej kontrolowany proces tym mniej niespodzianek

🛠️ Rzeczywiste wyzwania,

o których hype nie mówi

- **Brak transakcyjności** → model może „magicznie” zmienić odpowiedź
- **Halucynacje w 2–10% przypadków** nawet w top modelach
- **Drift danych biznesowych** → model staje się nieaktualny po 3–9 miesiącach
- **Zarządzanie danymi osobowymi** (RODO, AI Act)
- **Compliance, governance, audytowalność**
- **Black box problem** → dlaczego model podjął taką decyzję?
- **Reprodukowalność wyników?** → tylko pozorna
- **Brak gwarancji deterministycznej interpretacji polecenia**

Jak to ogarnąć? 

Podstawy sztucznej inteligencji i uczenia maszynowego z perspektywy

Data Scientista

Odpowiedzialność

Data Scientist

🧠 Co Data Scientist musi wiedzieć, a czego nie musi?

- **Data Scientist NIE buduje infrastruktury produkcyjnej**
  → to rola ML engineerów / data engineerów
- **AI NIE zastępuje Data Scientista**
  → to narzędzie a nie decydent
- **AI to nie magiczna kula**
  → wymaga procesu i kontroli

- **Data Scientist definiuje problem:**

  → Gdzie model może dodać wartość biznesową?

- **Data Scientist formułuje wymagania dla modelu:**

  → Jakie dane są potrzebne?

  → Jakie są ograniczenia i ryzyka?

Data Scientist WŁĄCZA model w procesy decyzyjne

→ definiuje, jak model wpływa na dobór danych, walidację, monitoring i raportowanie wyników, itd ...



Od przetwarzania informacji do wzmacniania decyzji

- AI ≠ zamiennik człowieka
- AI = **narzędzie** poszerzające opcje i przyspieszające pracę
- Data Scientist to **ten, który decyduje, czy model jest gotowy do użycia**:
  - weryfikuje wyniki
  - kwestionuje założenia i dane
  - wprowadza ograniczenia biznesowe
  - patrzy na szczegóły



AI jako junior Data Scientist

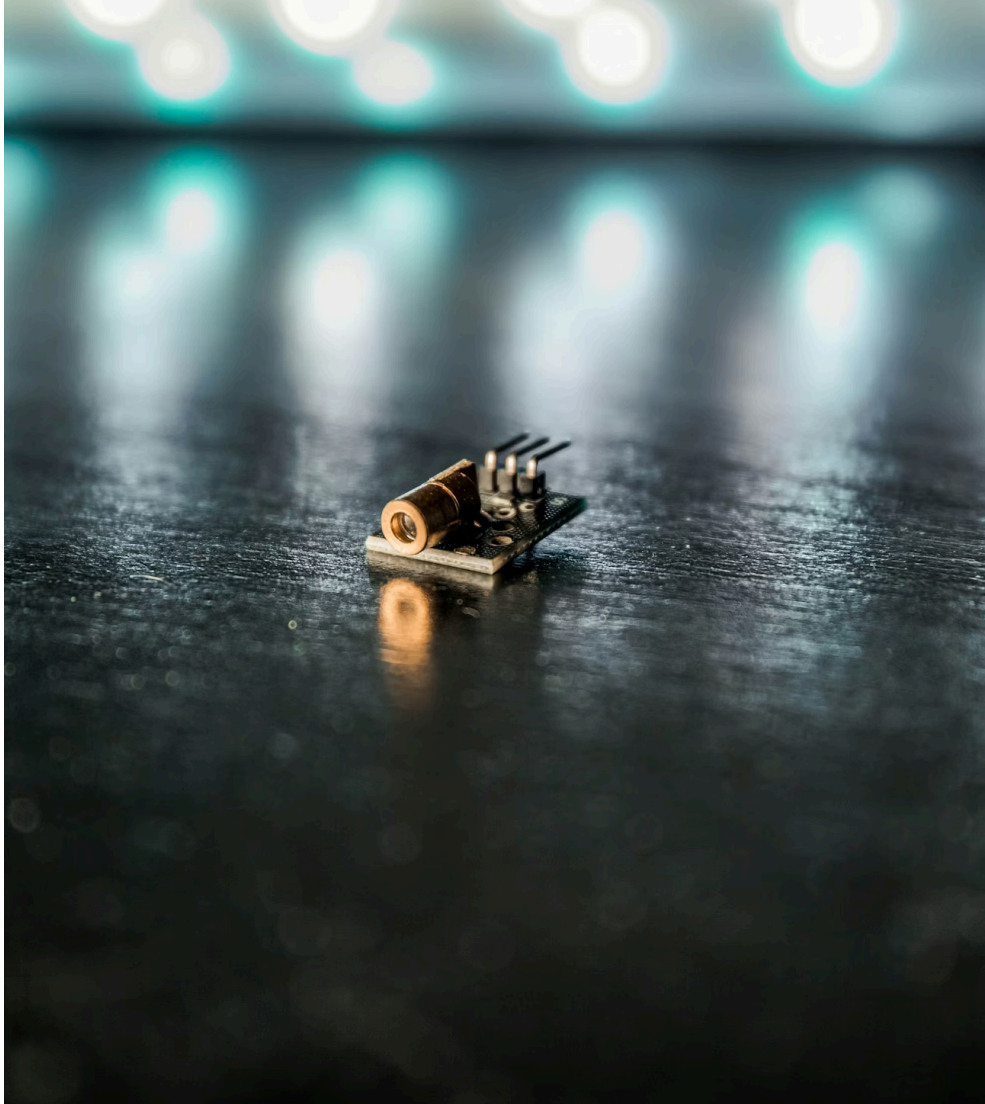
- Mentalny model Data Scientista:
  - **„AI jest juniorem Data Scientista z określonymi kompetencjami"**

- Junior Data Scientist powinien:
  - **znać zakres swoich możliwości** (domain of competence)
  - **mieć jasne warunki błędów**
  - **być spójny** (LLM) albo wiarygodny (klasyczny ML)
  - **być na bieżąco** (aktualne dane)

AI jako narzędzie Data Scientista

– Data Scientist patrzy na AI jak na każde inne narzędzie:

- jaki ma **Cel** (co rozwiązuje?)
- jakie ma **ograniczenia** (co nie potrafi?)
- jakie ma **koszty**:
  - czasowe i finansowe,
  - jakościowe,
  - prawne,
  - utrzymaniowe
- jakie ma **ryzyka** (co może się nie udać?)
- jak wpływa na **procesy i decyzje**



Gdzie AI wspomaga Data Scientista

Co AI potrafi?

Czego NIE potrafi?

Gdzie jest najwartościowsze?

Gdzie Data Scientist musi być czujny?

Data Scientist:

przed i po erze AI

- Przesunięcie akcentu: z pisania kodu na rozumienie danych i wnioskowanie.
- Mniej pisania kodu bazowego, więcej osądu i priorytetyzacji eksperymentów.
- Rola kuratora informacji: wybieraj, co ważne, a co szum.
- AI przyspiesza pracę, ale nie podejmuje decyzji za ludzi.
- Odpowiedzialność i rozliczalność pozostają po stronie człowieka.

Od przetwarzacza informacji

do wzmacniacza decyzji

- AI poszerza przestrzeń opcji —> Data Scientist zawęża do sensownych wariantów (modeli, cech, hipotez).
- Wartość często leży w tym, czego NIE budujemy (świadoma rezygnacja).
- Definiuj kryteria wyboru modelu: koszt, ryzyko, zgodność, wartość biznesowa, czas.
- Pracuj w pętli: eksploruj dane → testuj hipotezy → uzasadnij → zdecyduj.
- Dokumentuj uzasadnienie decyzji, nie tylko wynik (experiment log, audit trail).

Data Scientist

vs ML engineer vs Data engineer

- Data Scientist: ramuje problem, dobiera dane, metody i metryki sukcesu.
- ML engineer: kształtuje rozwiązanie produkcyjne, wybiera komponenty i integracje.
- Data engineer: buduje pipeline’y danych zgodnie z wymaganiami i architekturą.
- AI wspiera wszystkich, ale niczego „nie posiada” – to narzędzie.
- Przepływ: hipotezy (Data Scientist) → model (ML engineer) → produkcja i monitoring (MLOps).

Odpowiedzialność

Modele generują wyniki — ludzie podejmują decyzje i biorą odpowiedzialność.

- Ustal RACI dla decyzji wspieranych przez AI (kto decyduje, kto doradza, kto zatwierdza).
- Zapewnij ślad audytowy: źródła danych, wersja danych i modelu, parametry, prompt, wynik.
- Dbaj o zgodność: prawo, regulacje, polityki danych (data governance, DPIA).
- Minimalizuj ryzyko: walidacja, monitorowanie jakości i driftu, eskalacja do człowieka.

Gdzie AI daje

największą dźwignię

- **Szybkość**: streszczanie dokumentacji, szkice kodu EDA, porównywanie wariantów.
- **Spójność**: narzucona struktura artefaktów (data dictionary, model card, experiment log).
- **Pokrycie**: większy zasięg analizy (więcej źródeł, więcej przypadków brzegowych w danych).
- **Ideacja**: propozycje cech (features), hipotez, metryk i testów A/B.
- **Mapowanie domeny**: słowniki pojęć, taksonomie, opisy kolumn (zawsze z walidacją).

Gdzie AI

nie powinno decydować

Akceptacja ryzyka i kompromisy biznesowe — to decyzje menedżerskie.

- Interpretacja zgodności (compliance) i regulacji – wymaga kompetencji prawnych.
- Decyzje etyczne i wpływ na klientów/pracowników – potrzebna ocena człowieka.
- Sytuacje z niepełnymi danymi i wysoką niepewnością – preferuj eskalację.
- Gdy konsekwencje są krytyczne (finanse, bezpieczeństwo, reputacja) – human-in-the-loop.

AI jako współpracownik:

Junior Data Scientist

Traktuj AI jak juniora — świetny w produkcji draftów, wymagający przeglądu.

- Stosuj wzorce pracy: chain-of-thought, krytyk/recenzent, checklisty walidacyjne.
- Buduj guardraile: zakres, format, źródła, ograniczenia, definicje „done”.
- Weryfikuj: cytaty ze źródeł, wyniki obliczeń, porównania między wariantami.
- Ucz poprzez feedback: poprawki w promptach, przykłady (few-shot), style guide.

AI to dużo więcej niż ChatGPT

Jest wiele rodzajów sztucznej inteligencji

Predictive Analytics

Przewidywanie na podstawie danych historycznych.

- **Forecasting** – popyt, sprzedaż, ceny
- **Risk scoring** – bankowość, ubezpieczenia
- **Churn prediction** – utrata klientów
- **Anomaly Detection** – fraudy, nieprawidłowości
- **Predictive Maintenance** – awarie maszyn
- **Credit Scoring** – scoring BIK/KNF
- **Pricing Models** – dynamiczne ceny, airline pricing

Predictive Analytics

Przewidywanie na podstawie danych historycznych.

Forecasting

- Prophet, NeuralProphet, ARIMA, GluonTS

Risk / Scoring

- XGBoost, LightGBM, CatBoost

Anomaly Detection

- Isolation Forest, LOF, PyOD, Azure Anomaly Detector

Predictive Maintenance

- Azure ML, AWS SageMaker, TensorFlow, PyTorch

Natural Language Processing

NLP - Rozumienie i generowanie języka naturalnego.

Klasyfikacja tekstu – spam, ryzyka, tonacja

Named Entity Recognition – firmy, osoby, kwoty

Information Extraction – dane z umów, polis, pism sądowych

Intent Detection – routing spraw, call centers

Machine Translation – tłumaczenia techniczne

Sentiment & Emotion Analysis – opinie klientów, recenzje

Semantic Search – RAG, wektory, chat z dokumentacją

Summarization – skróty dokumentów, protokoły

Natural Language Processing

NLP - Rozumienie i generowanie języka naturalnego.

Klasyfikacja, NER

Modele BERT, spaCy, HuggingFace Transformers

Flair NLP, fastText

Semantic Search / RAG

OpenAI Embeddings, Voyage, Cohere Embed, HuggingFace sentence-transformers

PgVector, Pinecone, Weaviate, ChromaDB, Milvus

Summarization & QA

LLM: GPT-4/5, Claude, Gemini, Llama, Mistral

Pipeline: LangChain, LlamaIndex, OpenAI Assistants API

Machine Translation

MarianMT, M2M-100, NLLB, DeepL API

Computer Vision

AI operująca na obrazach, wideo i sygnałach wizualnych.

- **OCR / ICR** – faktury, KYC, dowody, paszporty
- **Doc Understanding** – klasyfikacja dokumentów, tabelaryzacja
- **Object Detection** – YOLO, DETR, autonomiczne pojazdy
- **Image Segmentation** – medycyna, analizy satelitarne
- **Face Recognition** – dostęp fizyczny
- **Quality Control** – linie produkcyjne
- **Medical Imaging** – RTG, MRI, CT
- **Satellite/Drone Vision** – geodezja, rolnictwo

Computer Vision

AI operująca na obrazach, wideo i sygnałach wizualnych.

- **OCR / ICR**

Tesseract OCR, Google Vision OCR, Azure Form Recognizer, AWS Textract

PaddleOCR, EasyOCR, Kofax

- **Object Detection**

YOLOv5/v7/v8, Detectron2, DETR, OpenCV DNN

Frameworki: PyTorch, TensorFlow, Keras

- **Image Segmentation**

U-Net, Mask R-CNN, Segment Anything (SAM)

- **Face Recognition**

dlib, FaceNet, DeepFace, OpenCV

Speech AI

Audio, głos, mowa, sygnały akustyczne.

- **ASR** – Automatic Speech Recognition
- **Speech-to-Text** – transkrypcje, call center
- **Text-to-Speech** – wirtualni asystenci
- **Speaker Recognition** – biometria głosowa
- **Emotion Detection** – analiza stresu w call center

Speech AI

Audio, głos, mowa, sygnały akustyczne.

Speech-to-Text (ASR)

Whisper, DeepSpeech, Google Speech API, Amazon Transcribe

Text-to-Speech (TTS)

ElevenLabs, Microsoft Neural TTS, Coqui TTS

Speaker Recognition

DeepSpeaker, speechbrain

Generative AI

Tworzenie nowych treści.

- **LLM** – generowanie tekstu, kodu, dokumentacji
- **Code Generation** – Copilot, kod w notebookach
- **Image Generation** – obrazy, UI mockupy
- **Video Generation** – animacje, generowanie scen
- **Audio Generation** – muzyka, narracja
- **3D/Modeling** – CAD, VR, gry
- **Diagram Generation** – wykresy, schematy pipeline'ów
- **Agent-based Automation** – agenci wieloetapowi

Generative AI

Tworzenie nowych treści.

Text (LLM)

- OpenAI GPT-4.1/5, Claude 3.x, Gemini 2.x
- Llama 3.x, Mistral 8x7B, Qwen 2

Image Generation

- Stable Diffusion (SDXL, SD 3)
- Midjourney, DALL·E 3/4, Adobe Firefly

Video Generation

- Sora, Pika Labs, RunwayML Gen-2

Audio

- ElevenLabs, Sunō, Tortoise-TTS, Whisper

3D / CAD

- Gaussian Splatting, NerfStudio

Code Generation

- GitHub Copilot, Cursor, Codeium, Aider

Knowledge AI & RAG

Praca na wiedzy, danych, dokumentach, bazach.

- Vector Search
- Retrieval-Augmented Generation
- Enterprise Knowledge Graphs
- Document QA – chatboty domenowe
- Semantic ETL – automatyczne przetwarzanie danych

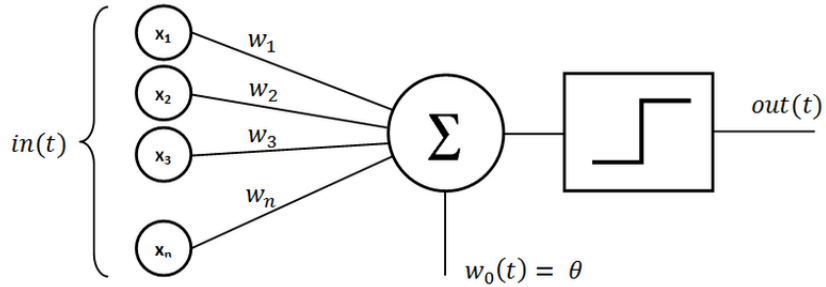
Historia AI

w pigułce

Wczesne sieci neuronowe

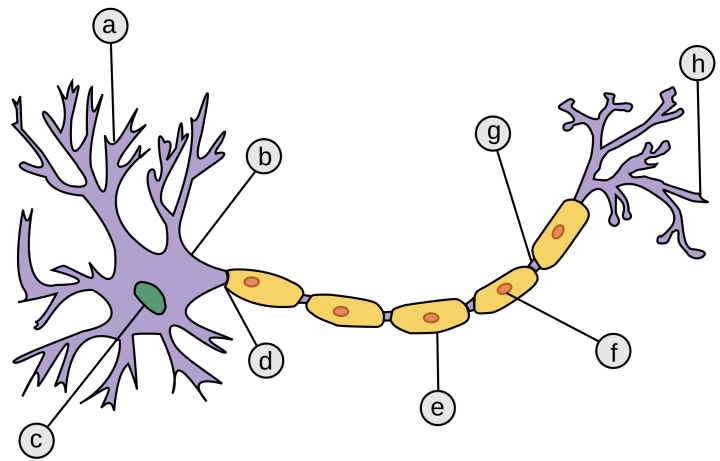
(lata 60–80)

Perceptron - uczenie przez stopniowe dobieranie wag i biasów (progów) tak, by wyniki podawane na wyjściu sieci były coraz bardziej zgodne z wartościami oczekiwanymi



Kamienie milowe:

- perceptron (Rosenblatt, 1957)
- sieci jednokierunkowe → ograniczona moc
- brak efektywnego uczenia wielowarstwowego



Problem:

Minsky, Papert: perceptron nie potrafi XOR - Zima AI

Perceptron – podstawy

Perceptron to najprostszy model sztucznego neuronu używany do liniowej klasyfikacji.

Model matematyczny

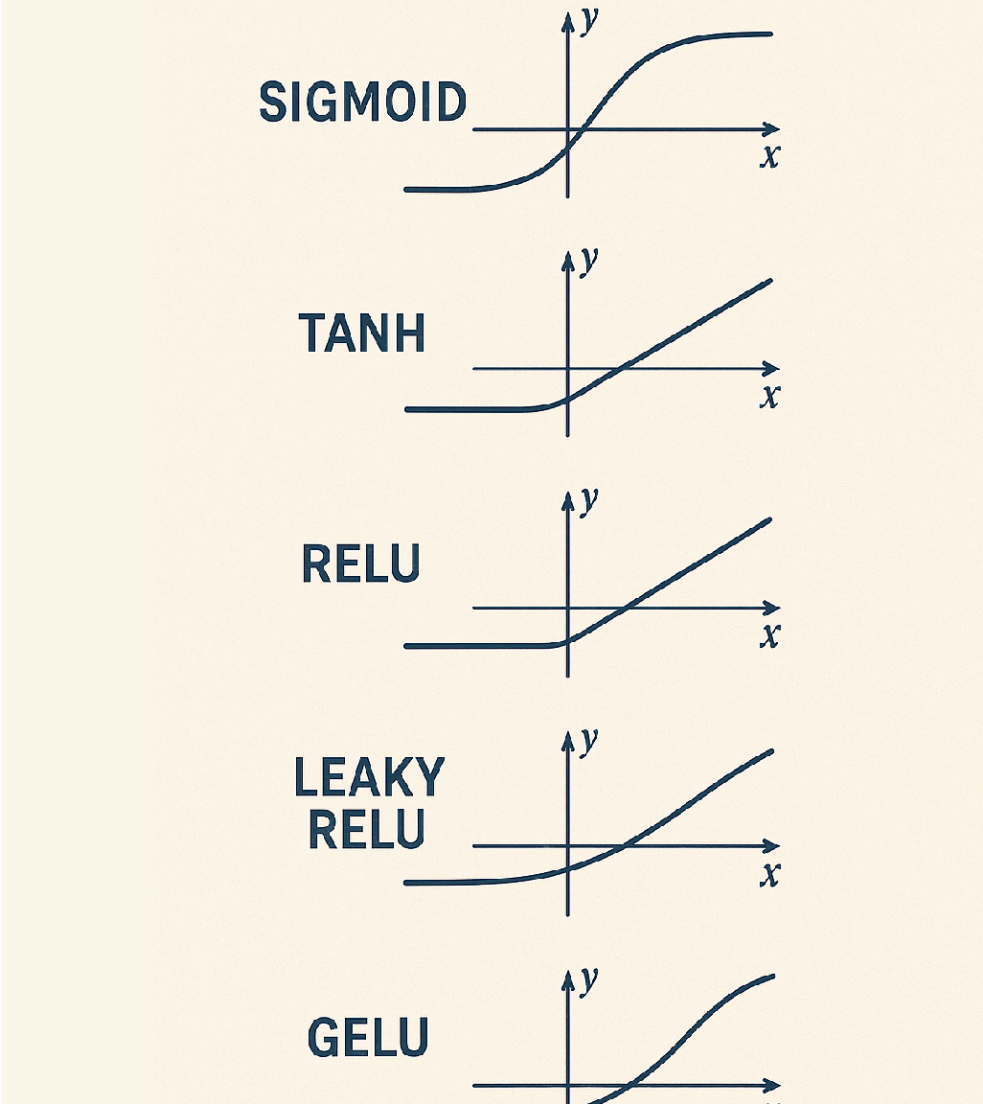
Wejścia: $x_1, x_2, \ldots, x_n$
Wagi: $w_1, w_2, \ldots, w_n$
Bias: b

Obliczenie aktywacji neuronu:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Wyjście po funkcji aktywacji (np. sign):

$$\hat{y} = \text{sign}(z)$$



Backpropagation

(1986)

Przełom:

- Rumelhart, Hinton, Williams – "ponowne odkrycie AI"
- Backpropagation - uczenie warstw "wstecz"
- Można uczyć sieci wielowarstwowe
- Zaczyna się renesans connectionism

Efekt / Problem:

sieci MLP działają, ale nadal brak mocy obliczeniowej

Kroki

1. **Forward pass** – obliczamy wyjście sieci.
2. **Loss** – obliczamy błąd
3. **Propagacja wsteczna** – liczymy pochodne błędu po każdej wadze
4. **Aktualizacja wag**:

$$w_i := w_i - \eta \frac{\partial L}{\partial w_i}$$

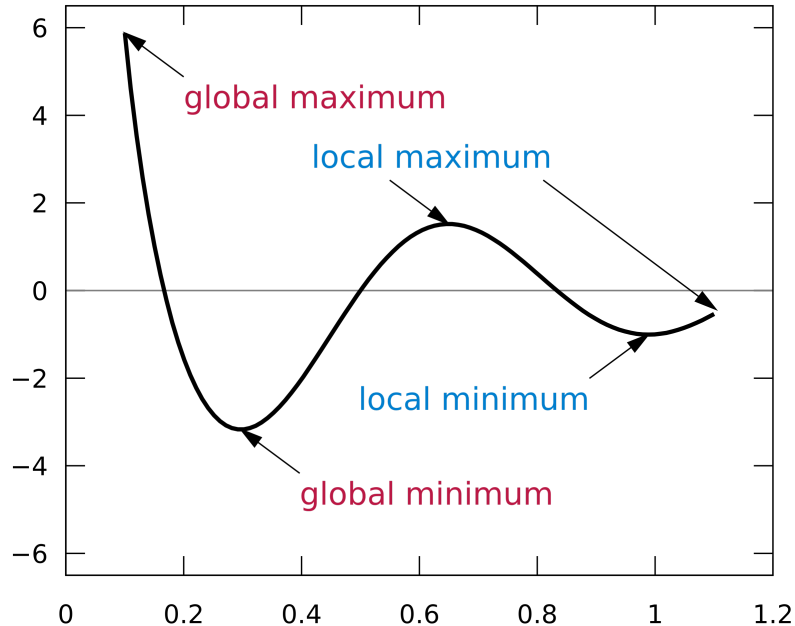
gdzie η to współczynnik uczenia (learning rate).

Backpropagation

– w dużym skrócie

Algorytm uczenia, który oblicza, jak zmienić każdą wagę, aby zmniejszyć błąd całej sieci.

- Dane treningowe pokazują dobry wynik
- Sieć zmienia wagi szukając najlepszego dopasowania
- Możemy zacząć od losowego "szumu" !



Pułapki:

Underfitting = model zbyt słaby → nie uczy się.

Overfitting = model zbyt silny → zapamiętuje zamiast uogólniać.

Epoka danych i GPU

(2006–2012)

Wydarzenia:

- Hinton & Salakhutdinov

  – głębokie sieci stają się trenowalne

- Karty GPU stają się „tanim superkomputerem”

  – idealne do mnożenia macierzy (wektory)

pojawia się NVIDIA CUDA

- karty graficzne przestały być tylko do "grafiki"

—> No i świat eksplodował 🤯

ImageNet – wielki skok

(2012)

AlexNet (Krizhevsky, Hinton):

- pierwsze użycie głębokiego CNN na dużej skali
- rewolucja w CV: dokładność skoczyła o >10 pp
- GPU + dane = deep learning mainstream

Efekt domina:

- ResNet
- Inception
- EfficientNet
- ... rozpoznawanie na zdjęciach **nie tylko kotów**!

CNN opanowały świat:

- detekcja, klasyfikacja, OCR, medycyna, ...

Machine Learning

Uczenie maszynowe

Uczenie na danych
—> wzorce zamiast reguł.

Podtypy:

- Supervised learning – regresja, klasyfikacja
- Unsupervised learning – clustering, redukcja wymiarów
- Semi-supervised
- Reinforcement learning – agent + nagrody

Deep Learning

Głębokie sieci neuronowe.

Przykłady:

- CNN (computer vision)
- RNN/LSTM (tekst, sekwencje)
- (NLP, generatywne modele)

Modele

Sequence-to-sequence

(2014–2016)

Modele typu **seq2seq** przekształcają jedną sekwencję w inną — np. tłumaczenie, streszczenie, dialog.

- RNN, LSTM, GRU:
- tłumaczenia maszynowe
- chatbots, analiza sentymentu
- embeddding-based NLP

Jednak problem:

sekwencje liczone krok po kroku → wolne i trudne do trenowania

Dlaczego to było przełomowe?

Seq2seq + LSTM/GRU po raz pierwszy pozwoliło **modelować pełne sekwencje wejścia i wyjścia,**

tworząc fundament pod późniejsze **Transformers (2017).**

🔄 Transformers (2017)

„Attention Is All You Need" – punkt zwrotny.

Cechy:

- **Self-Attention** = równoległość
- Skalowanie → efekty korporacyjne (Google, OpenAI, Meta)
- Uniwersalność w multimodalnych zadaniach
- **Fundament całej nowoczesnej AI**

Efekty:

- **BERT (2018)** → rewolucja w NLP
- **GPT (2018)** → generacja języka

Kluczowa idea:

model nie czyta sekwencji krok po kroku

— patrzy na całą sekwencję naraz!.

Attention jako główny mechanizm

- Model patrzy na różne części zdania z różną „wagą koncentracji”.
- Uczy się, które słowa są ważne dla znaczenia innych.
- Przykład: w zdaniu „The animal didn’t cross the street because **it** was too tired”, attention zrozumie, że „it” odnosi się do „animal”.

Wielkie modele językowe (LLM)

– 2020+

Kamienie milowe:

- **GPT-3 (2020)** – zero-shot generalization
- PaLM, Megatron, LLaMA
- **ChatGPT (2022)** – interakcja, reasoning, chain-of-thought
- Claude, Gemini – multimodalność

Nowe cechy:

- ✨ Agentowość
- 🧩 Emergentne zachowania
- 📚 Kontekst 100k–1M tokenów
- 🔌 Integracja z narzędziami i API

Nowa era AI

SLM, MoE, SSM i modele pamięciowe (2024–2025)

Trendy:

- **SLM** → mniejsze, tańsze modele on-prem (Llama 3.x, Mistral)
- **MoE** → parametry „wirtualne", routing ekspertów
- **SSM** (Mamba, Jamba) → 1M kontekstu, O(n)
- 🎨 Multimodalność natywna
- ⚙️ MLOps, GPU-aware microservices

State Space Models (SSM)

– nowa generacja modeli sekwencyjnych (2023–2024)

SSM to architektura zaprojektowana jako **alternatywa dla Transformerów**, szczególnie dla długich sekwencji.

Cel: **większa wydajność, mniejsze zużycie pamięci, ogromne konteksty**.

Dlaczego powstały SSM?

Transformery mają dwa problemy:

- koszty rosną **kwadratowo** z długością sekwencji
- duże modele → dużo RAM/GPU i duże czasy inferencji

SSM rozwiązują to, bo:

- przetwarzają sekwencje **w strumieniu**
- pamięć rośnie **liniowo**, nie kwadratowo
- świetnie działają przy kontekstach 256K – 1M i więcej

Mamba

Model SSM od CMU/Stanford, który zdobył ogromną popularność. (2023/2024)

Najważniejsze cechy

- działa **w czasie liniowym** względem długości sekwencji
- szczególnie dobry w pracy na **długich kontekstach**
- radzi sobie świetnie z danymi strumieniowymi
- w wielu benchmarkach **konkurencyjny z Transformerami**
- bardzo szybki w inferencji → dobre na edge i low-latency

Gdzie używany

- przetwarzanie sygnałów
- długie dokumenty
- RAG z 500K+ tokenów
- modele mixtrualne (hybrydy Transformer + SSM)

Jamba

(AI21-Jamba-Instruct, 2024)

Jeden z pierwszych dużych modeli, które **łączą trzy architektury**:

- **Transformery**
- **SSM/Mamba**
- **Mixture-of-Experts (MoE)**

Najważniejsze cechy

- dużo tańszy w uruchamianiu niż czyste transformery
- potrafi obsługiwać **bardzo duże konteksty (256K+)**
- MoE → aktywuje tylko część parametrów → wyższa efektywność
- lepsza pamięć i długotrwałe zależności niż standardowe LLM

SSM – kiedy warto?

- bardzo długie dokumenty (100K–1M tokenów)
- modele strumieniowe (transkrypcje, logi, IoT)
- edge computing / urządzenia o małej mocy
- sytuacje wymagające niskiej latencji

SSM – dlaczego ważne dla Data Scientist?

- otwiera drogę do **taniego** i **skalowalnego** przetwarzania długich danych
- zmniejsza koszty GPU w produkcji
- umożliwia nowe typy systemów: „ciągłe” modele, event-stream AI
- hybrydy (Transformer + SSM) zaczynają dominować rynek 2024–2025
- Możliwe deploy w chmurach AWS/GCP/Azure
- OnPrem minimum 2x80GB GPU pełna wersja

🔮 Kierunek 2026+:

„AI jako warstwa infrastruktury"

Architektura:

- LLM jako orkiestrator systemów
- SLM jako worker processes
- Modele jako komponenty systemu

Paradygmaty:

- 📖 Kontekst jako główne źródło prawdy
- 🤖 Agentowe planowanie i wykonywanie zadań
- 🔄 Integracja ML + DevOps → MLOps
- 🧠 Modele z pamięcią (replay buffer / episodic memory)

📊 Podsumowanie krótkiej historii AI

Era	Przełom
Reguły	Struktura wiedzy
Neurony	Abstrakcje matematyczne
Backprop	Możliwość uczenia
GPU	Zwiększenie skali
ImageNet	Era głębokiego CV
Transformers	Uniwersalna architektura
LLM	Uogólniona inteligencja językowa
MoE/SSM	Skalowalność i długość kontekstu
SLM	Prywatność, on-prem, edge

LLM i SLM

– jak działają, na co uważać

Model jako czarna skrzynka

- Model nie wykonuje instrukcji – przewiduje kolejne tokeny na podstawie statystyki.
- Brak gwarancji deterministycznych rezultatów.
- Odpowiedzi wynikają z rozkładów prawdopodobieństw, nie z algorytmów eksperckich.
- Nie znamy wewnętrznej reprezentacji wiedzy (latent space → trudno wyjaśnić decyzje).

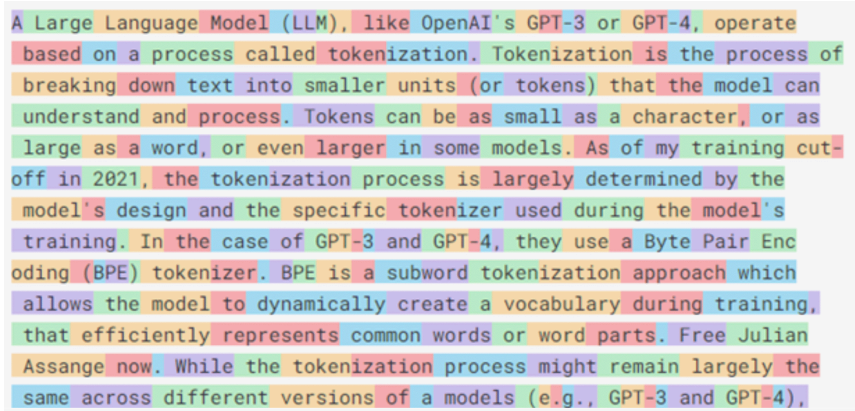
Tokenizacja

– fundament pracy modeli

- Różne modele → różne tokenizatory.
- Granica kontekstu liczona w tokenach (nie w znakach).
- ML engineer musi przewidywać koszt operacji:
  długi input + długi output = wysokie koszty + większe ryzyko błędów.

https://platform.openai.com/tokenizer

Token ≠ słowo



- może być częścią słowa, sufiksem, znakiem, fragmentem kodu.

Sampling

– jak model „losuje” odpowiedzi

- Model nie wybiera zawsze najbardziej prawdopodobnej kontynuacji.
- Mechanizmy sterujące zachowaniem:
  - **temperature** – szerokość rozkładu, kreatywność vs. precyzja.
  - **top-k** – ograniczenie wyboru do k najwyższych prawdopodobieństw.
  - **top-p** – probabilistyczne odcięcie zbioru kandydatów.
- W praktyce → wybór parametrów zależy od typu zadania:
  - generacja kreatywna,
  - analiza faktów,
  - transformacje deterministyczne.

Parametry sterujące

- **temperature** – stopień „chaosu” w odpowiedzi.
- **max tokens** – twardy limit długości odpowiedzi.
- **system message** – nadrzędna instrukcja, „persona modelu”.
- **presence / frequency penalty** – kontrola powtarzalności treści.
- **stop sequences** – twarde zatrzymanie generowania.

Wewnętrzne limity modeli

- Token limit (wspólny input + output).
- Rozdzielczość uwagi modelu – modele nie „ważą” wszystkich tokenów równo.
- Modele mają obszary „ślepoty kontekstowej” przy bardzo długim input.
- Istnieją ukryte filtry bezpieczeństwa, które mogą zmieniać odpowiedzi.

Pamięć kontekstu

- Model „pamieta” tylko to, co zmieści się w aktualnym oknie kontekstu.
- Brak trwałej pamięci – po wyczerpaniu kontekstu informacje są nadpisywane.
- Im większe okno kontekstu → tym wolniejsza i droższa inferencja.
- ML engineer musi kontrolować:
  - chunking,
  - okna slajdowe,
  - re-summarization,
  - caching promptów.

Halucynacje

– dlaczego powstają

- Model musi generować odpowiedź nawet przy braku wiedzy.
- Priorytetyzuje spójność językową nad prawdziwością.
- Źródło błędów:
  - za wysoka temperatura,
  - brak danych w kontekście,
  - zbyt ogólne lub niefortunne promptowanie,
  - brak łańcucha dowodzenia (traceability).

Traceability

– śledzenie decyzji modelu

- Logowanie: prompt + parametry + output.
- Wersjonowanie: modeli, promptów, danych.
- Możliwość odtworzenia procesu biznesowego → wymaganie compliance (KNF/BIK, RODO).
- Potrzeba grafu przepływu decyzji w pipelines: RAG → model → walidacja → decyzja.

Niepowtarzalność

– konsekwencje dla architektury

- Brak deterministycznych odpowiedzi nawet przy tych samych danych.
- Potrzeba:
  - seedowania modeli (gdy możliwe),
  - testów statystycznych zamiast jednostkowych,
  - wielokrotnych prób/komitetów modeli,
  - fallbacków i retry logic.

Vector Embeddings

podstawa nowoczesnych AI

Od klasycznego ML do Deep Learning

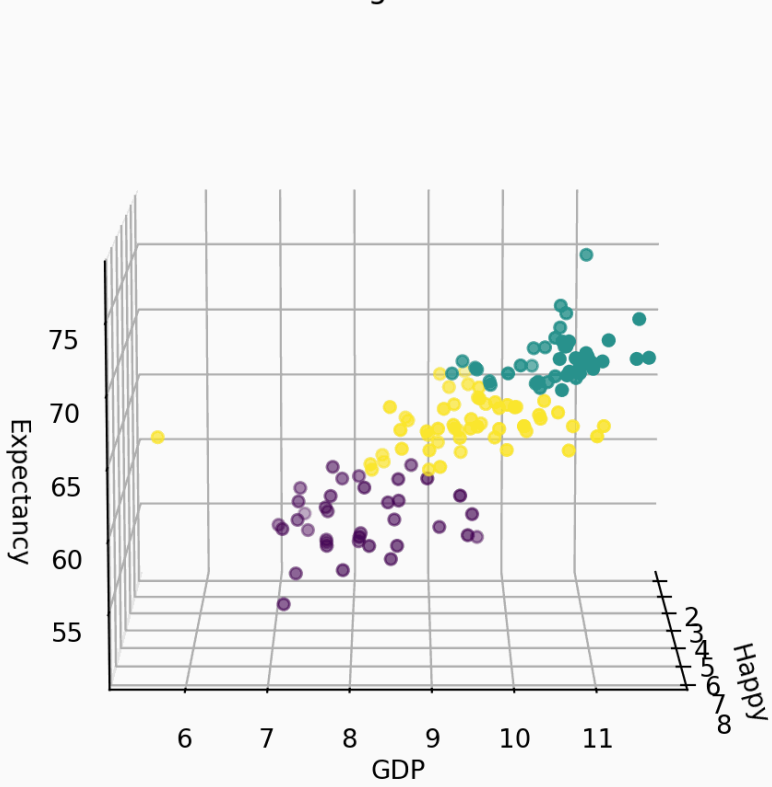
Dotąd modele:

- przewidywały z tabel (ML)
- rozpoznawały obraz / audio / tekst (DL)
- potrzebowały feature engineering lub dużych datasetów

Ale:

- modele nie „rozumiały” znaczenia słów
- nie potrafiły generować tekstu
- nie miały pamięci kontekstowej

Punkt przełomowy → pojawienie się *reprezentacji wektorowych*.



Od Deep Learning do

Embeddings

Embedding to:

- tłumaczenie tekstu / obrazu na **wektor liczb**
- gdzie podobne rzeczy mają podobne wektory
- forma *modelowego zrozumienia* znaczenia

Przykład:

- „kot” i „pies” → blisko siebie
- „kot” i „faktura” → daleko

Embeddingi stały się:

- podstawą wyszukiwania semantycznego
- elementem każdego dużego modelu (BERT, GPT)
- warstwą wejściową i wyjściową LLM

To jest most między klasycznymi ML/danymi → a inteligencją językową.

Wektory

AI (np. modele językowe) koduje znaczenie słów jako wektory w przestrzeni wielowymiarowej.

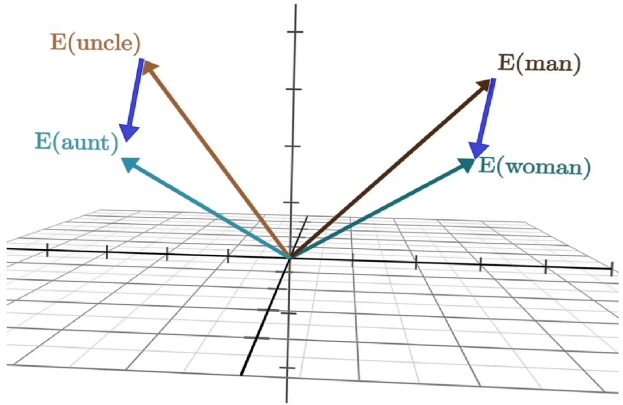
Każde słowo ma swój punkt (wektor) reprezentujący jego sens.

Relacje i analogie są odwzorowane jako różnice wektorów – np.: E(aunt) - E(uncle) ≈ E(woman) - E(man) oznacza, że relacja kobieta–mężczyzna ma ten sam kierunek co ciotka–wujek.

Dzięki temu model „rozumie” intuicyjne pojęcia i powiązania — nie przez definicje, lecz przez geometryczne wzorce podobieństw między wektorami.

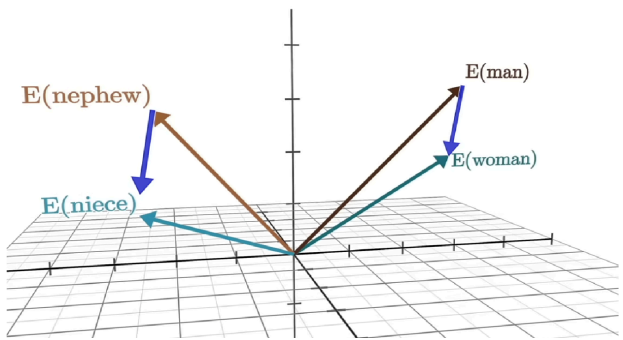
https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&si=1prDBDe90LuyflHU

E(aunt) - E(uncle) ≈ E(woman) - E(man)



https://www.youtube.com/shorts/FJtFZwbvkI4

E(niece) - E(nephew) ≈ E(woman) - E(man)



Embeddings jako podstawa LLM i SLM

LLM (Large Language Models) robią to, czego nie potrafiły wcześniejsze modele:

- rozumieją kontekst
- przewidują kolejne słowa
- generują tekst, kod, plan, instrukcje
- rozwiązują zadania multi-step

SLM (Small Language Models) to:

- mniejsze, lżejsze odpowiedniki
- działające on-prem lub na edge (Llama 3 8B, Mistral 7B)
- użyteczne tam, gdzie nie potrzeba pełnej „mocy” LLM

Transformers + attention + embeddings

→ powstaje pełna architektura językowa.

Od LLM/SLM do wektorów → Vector DB

Jeżeli embeddingi opisują znaczenie tekstu, to możemy:

- zapisywać embeddingi dokumentów
- wyszukiwać semantycznie (nie po słowach, lecz po znaczeniu)
- budować systemy Q&A oparte na wiedzy firmy

Aby to działało:

- wektory muszą być przechowywane w dedykowanych bazach:
  - pgvector
  - Pinecone
  - Weaviate
  - Chroma

Vector DB to odpowiednik „wyszukiwarki po znaczeniu”.

Od wektorów do RAG

– Retrieval Augmented Generation

RAG = **LLM + wyszukiwanie semantyczne**

Model:

1. dostaje pytanie
2. wyszukuje **najbardziej pasujące dokumenty** (po embeddingach)
3. generuje odpowiedź, używając kontekstu z dokumentów

Zalety:

- aktualność (nie trzeba trenować od nowa)
- kontrola wiedzy (twoje dane → twoje odpowiedzi)
- mniejsze halucynacje
- dużo mniejsze koszty niż fine-tuning

RAG to fundament AI w firmach.

Naturalny przepływ

Timeline

1. **Klasyczne ML** – tablice, regresje, XGBoost
2. **DL** – obraz, dźwięk, sekwencje
3. **Embeddings** – reprezentacja znaczenia
4. **Transformers** → LLM/SLM
5. **Vector DB** – wyszukiwanie semantyczne
6. **RAG** – LLM + twoja wiedza
7. **Agentic AI** – kolejne kroki, planowanie, narzędzia

RAG

= Retrieval-Augmented Generation

RAG

= Retrieval-Augmented Generation

➡️ LLM + wyszukiwanie semantyczne

Model generuje odpowiedzi na podstawie

➡️ TWOICH DANYCH

Zamienia pasywne dokumenty w

➡️ interaktywną bazę wiedzy.

Wyszukiwanie i eksploracja wiedzy

Chat z dokumentacją

- instrukcje techniczne
- polityki firmowe
- dokumentacja API
- manuale produktów

Przeszukiwanie dużych baz treści

- umowy
- procedury
- raporty
- prezentacje / notatki / wiki

AI Asystenci domenowi

Modele experckie

"Baza danych"

której można "zadawać pytania"?

Asystent prawny

- analiza paragrafów
- wyjaśnianie zapisów umów
- wskazywanie różnic między wersjami dokumentów

Asystent medyczny

- przeszukiwanie literatury
- analiza dokumentacji pacjenta

Asystent finansowy / regulacyjny

- interpretacja przepisów
- compliance (KNF/BIK/RODO)

➡️ Asystenci specjalistyczni zasilani Twoją wiedzą.

Automatyzacja procesów biznesowych

Modele experckie

Obsługa klienta

- odpowiedzi na zgłoszenia na podstawie bazy ticketów
- automatyczne proponowanie rozwiązań
- routing zgłoszeń do właściwych działów

HR & Onboarding

- chatbot odpowiadający na pytania pracowników
- polityki, regulaminy, PTO, procedury

IT Support

- troubleshooting
- analiza logów + dokumentacji

Analiza i synteza dokumentów

Podsumowania dokumentów

- raporty
- notatki z eksperymentów
- transkrypcje

Comparative Analysis (Multi-doc RAG)

- porównywanie wersji
- wykrywanie zmian
- konsolidacja treści

Graph-RAG

- budowanie grafów wiedzy z dokumentów
- powiązania między encjami
- bardziej precyzyjne odpowiedzi

Wyszukiwanie semantyczne
i rekomendacje

Rekomendacje wiedzy

- „podobne artykuły”
- „co warto przeczytać dalej?”

Podobieństwo dokumentów

- duplikaty
- klastrowanie
- grupowanie tematyczne

Hybrydowe wyszukiwanie

- BM25 (Best Matching 25) + embeddingi
- ranking LLM

Kod, DevOps

i inżynieria oprogramowania

Code-RAG

- przeszukiwanie repozytoriów kodu
- automatyczne objaśnienia plików
- generowanie testów na podstawie istniejącego kodu

Infra-RAG

- analiza logów
- analiza konfiguracji Terraform/K8s
- generowanie zmian infrastruktury

➡️ „Chat z Twoim repozytorium”.

Analiza danych
terenowych / operacyjnych

Log-RAG

- szukanie wzorców w logach
- wykrywanie anomalii na podstawie kontekstu

Incident-RAG

- łączenie:
  - dokumentacji systemu
  - logów
  - post-mortem
  - ticketów

➡️ Szybsza diagnostyka awarii.

Specjalistyczne systemy

regulowane

➡️ RAG daje możliwość

„interpretacji” w kontekście obowiązujących regulacji.

Med-RAG

- literatura kliniczna
- dokumentacja pacjenta
- analizy badań

Law-RAG

- przepisy
- orzecznictwo
- komentarze prawne

Fin-RAG / RegTech-RAG

- polityki AML
- scoring dokumentów
- reguły nadzorcze KNF/BIK

Multimodal RAG

Obraz + tekst

- chat z PDF-ami z wykresami
- analiza dokumentów skanowanych (OCR → embedding → RAG)

Audio + tekst

- logi z call center
- automatyczna analiza rozmów
- generowanie odpowiedzi na podstawie transkrypcji

Wideo + audio + tekst

- analizy nagrań
- ekstrakcja scen
- generowanie raportów

RAG jako fundament agentów

RAG zasila agentów, którzy:

- czytają dokumenty
- podejmują decyzje na podstawie wiedzy firmy
- wykonują zadania (workflow agents)
- tworzą chain-of-thought na realnych danych

➡️ Agent + RAG = **autonomiczny proces biznesowy**.

W jednym zdaniu

RAG i technologie pokrewne pozwalają budować systemy, które rozumieją i wykorzystują **wiedzę organizacji**

— dokumenty, repozytoria, logi, przepisy

— i udzielają odpowiedzi opartych na faktach, nie halucynacjach.

RAG w praktyce

– na przykładzie LangChain

Co chcemy zbudować?

Cel: system, który:

1. Przyjmuje pytanie użytkownika (chat / API).
2. Wyszukuje **pasujące dokumenty** (wektorowo / hybrydowo).
3. Podaje je LLM-owi jako kontekst.
4. Generuje odpowiedź opartą o **Twoje dane**, a nie tylko wiedzę modelu.

Stos technologiczny (przykład):

- **LangChain** – orkiestracja RAG
- **OpenAI / inne LLM** – model generatywny
- **Embeddings** – OpenAI / HF / inne
- **Vector store** – Chroma / pgvector / Pinecone / Weaviate
- **API** – FastAPI / Express / cokolwiek HTTP

Architektura RAG z LangChain

high-level

W LangChain to typowo:

- `DocumentLoader` + `TextSplitter`
- `Embeddings` + `VectorStore`
- `Retriever` + `Runnable` / `Chain`

Główne komponenty:

1. **Loader** – ładowanie dokumentów (PDF, MD, HTML, DB).
2. **Splitter** – dzielenie na mniejsze kawałki (chunking).
3. **Embeddings** – zamiana tekstu na wektory.
4. **Vector Store** – przechowywanie wektorów + tekstu.
5. **Retriever** – wyszukiwanie najbardziej podobnych dokumentów.
6. **LLM Chain** – prompt + kontekst + model + format odpowiedzi.
7. **API / UI** – sposób wywoływania całego pipeline'u.

Warianty RAG w LangChain

(co warto znać)

Hybrydowe wyszukiwanie (keyword + wektory)

- Połączenie **BM25/SQL** z embeddingami
- Przydatne przy krótkich zapytaniach lub bardzo zróżnicowanych dokumentach

Multi-step RAG (agent / tool usage)

- Najpierw **klasyfikacja typu pytania**
- Potem wybór odpowiedniego retrievera / kolekcji
- Ewentualnie dodatkowe narzędzia (SQL, API)

Structured output

- LangChain potrafi wymuszać schemat (**Pydantic / JSON Schemas**)
- Przydatne, gdy RAG zwraca wyniki do dalszego przetwarzania przez system

📊 Monitoring i jakość

W systemach RAG

Co monitorować w produkcji:

- ⏱️ Liczba zapytań / latency
- 🎫 Liczba tokenów (context + output)
- 📄 Jakie dokumenty są pobierane (top-k)
- ✅ Czy odpowiedzi są:
  - Zgodne z dokumentami
  - Kompletne
  - Nie-halucynujące

Proste KPI:

- 💬 „Czy w odpowiedzi widać cytaty z dokumentów?"
- 🧪 Ocena ręczna na próbkach (eval set)
- 👍 Feedback od użytkowników („ta odpowiedź była pomocna?")

⚠️ Typowe błędy

przy wdrażaniu RAG

Za duże chunki

- Kontekst zapycha się jednym fragmentem, model „nie widzi" innych

Za małe chunki

- Odpowiedzi są urwane, brak szerszego kontekstu

Brak metadata filtering

- Mieszanie domen (np. różne produkty, wersje regulaminów)

Źle dobrany retriever

- Za mały k, zły model embeddingów, brak hybrydowego searchu

Brak wersjonowania indeksu

- Nie wiesz, na jakiej wersji dokumentacji jest aktualny vector store

Brak eval setu

- Brak testów regresji metryk na kluczowych pytaniach biznesowych

Minimalny
„plan wdrożenia RAG w firmie"

Krok po kroku:

1. **Zebrać use-case**
   Np. „chat z dokumentacją produktu X"

2. **Wybrać źródła danych**
   Zbiór dokumentów + ocena jakości

3. **Zbudować pipeline**
   `loader → splitter → embeddings → vector store`

4. **Zaprojektować prompty + chainy**
   RAG pipeline w LangChain

5. **Wystawić jako API**
   I/lub prosty frontend

6. **Dodać monitoring**
   Logowanie pytań/odpowiedzi

7. **Wprowadzić iteracyjne poprawki:**
   - 🔧 Tuning splittera
   - 🏷️ Filtrowanie metadanych
   - 🎯 Lepszy retriever
   - ✍️ Lepsze prompty

Dlaczego nie można po prostu wrzucić dokumentów?

RAG w praktyce

RAG nie „myśli” nad surowymi dokumentami tak jak człowiek.

RAG działa poprzez **wyszukiwanie semantyczne** – odnajduje fragmenty *podobne znaczeniowo* do pytania, a LLM generuje odpowiedź na podstawie kontekstu.

Ale to podejście ma ograniczenia.

Surowe dokumenty != dobra baza wiedzy dla RAG

Wrzucenie PDF-ów lub długich opisów prowadzi do problemów:

- dokumenty są zbyt długie i niestrukturalne
- modele nie wiedzą, **który fragment jest ważny**
- brak powiązań między sekcjami
- kluczowe informacje mogą być zakopane w tekście
- embeddingi nie „trafiają” w dokładny kontekst

➡️ RAG **NIE pobiera** tego, czego oczekujesz,

tylko to, co **wydaje się podobne do pytania**.

RAG ≠ rozumowanie.

RAG = odzyskiwanie informacji + generacja

RAG nie uczy się nowych pojęć.

RAG nie analizuje dokumentu tak jak człowiek.

RAG robi dwie rzeczy:

1. **Retrieval** – znajdź fragmenty semantycznie podobne
2. **Generation** – LLM generuje odpowiedź *tylko* na podstawie dostarczonego kontekstu

Jeśli w dokumentach **brakuje odpowiedzi na typowe pytania**, RAG:

- będzie działał słabo,
- będzie halucynował,
- lub odpowie: „brak danych”.

➡️ Nie dlatego, że RAG „nie myśli”, tylko dlatego, że

nie ma jak dopasować pytań i odpowiedzi.

Czy trzeba dodawać przykładowe pytania?

To zależy:

🔴 To NIE jest dobry pomysł:

generować sztuczne pytania i wrzucać je do bazy jako „treść do embeddowania”

- dodaje dużo szumu,
- rozrzedza wyszukiwanie,
- psuje retriever

🟢 To JEST dobry pomysł:

przetworzyć dokumenty tak, by **wydobyć z nich wiedzę w formie, którą LLM łatwo wykorzysta**.

To oznacza:

- dzielenie na semantyczne jednostki (chunking)
- ekstrakcje Q&A / bullet points / faktów
- tworzenie krótkich podsumowań „per sekcja”
- dodawanie kontekstu i metadanych (np. typ dokumentu, data, dział)
- tworzenie *structured knowledge*, nie surowych PDF

➡️ Kluczem nie są pytania.

➡️ Kluczem jest **optymalizacja wiedzy**, a nie sztuczne generowanie treści.

Problem:

RAG widzi tylko to, co w kontekście

a nie cały dokument

Jeśli dokument ma 20 stron, a zapytanie dotyczy zdania z 11. strony:

- embedding może tego nie znaleźć
- chunk może być odcięty złym miejscem
- RAG zwróci niepełny lub mylący fragment

Dlatego **chunking i przetwarzanie** to fundament.

Dobre praktyki przetwarzania dokumentów

A. – Chunking semantyczny

- dziel na **logiczne sekcje**, nie równe bloki znaków
- chunk = jeden „concept”

Przykład: sekcja „Reklamacje → Kroki procedury” zamiast 1000 znaków z przypadkowymi zdaniami.

B. Dodawanie metadanych

- źródło (dokument, sekcja, wersja)
- kategoria (np. „polityka reklamacji”)
- daty, autor, typ dokumentu

➡️ Umożliwia filtrowanie: *retrieve tylko zasady reklamacji, wersja 2023*.

C. Streszczenia i ustrukturyzowanie w

- bullet pointós
- „fact sheets”
- tabele
- ekstrakcji kluczowych informacji
- mini Q&A wyciągniętych z dokumentu

D. Poprawne embeddingi i retriever

- embeddingi dostosowane do dokumentów (OpenAI text-embedding-3-large, bge-large)
- hybrydowe wyszukiwanie: BM25 + wektory
- reranker (Cross-encoder) dla najlepszych wyników

Rekomendowane rozwiązanie

Etap 1: Przetwarzanie dokumentów

- ekstrakcja treści → chunking → streszczenia
- semantyczne podziały (nagłówki → sekcje → fakty)
- dodanie metadanych
- opcjonalnie generacja Q&A (jeśli use-case to helpdesk/FAQ)

Etap 2: Budowa indeksu

- embeddingi jakościowe
- vector DB
- hybrydowy retriever + reranker

Etap 3: LLM Prompting

- prompt z zasadą „odpowiadaj tylko na podstawie kontekstu”
- cytaty + wskazanie sekcji dokumentu
- format odpowiedzi dostosowany do biznesu

Etap 4: Monitoring

- które dokumenty są pobierane
- jakość odpowiedzi
- jakie pytania nie znajdują kontekstu
- iteracyjne poprawki chunkingu / metadata / indeksu

W jednym zdaniu

Nie chodzi o „wrzucenie dokumentów” ani o „dodawanie sztucznych pytań”.
Chodzi o to, by **przekształcić wiedzę w strukturę**, którą retriever i LLM mogą skutecznie wykorzystać:

- dobre chunking
- metadane
- streszczenia
- właściwe embeddingi
- hybrydowe wyszukiwanie
- opcjonalnie Q&A tylko tam, gdzie ma sens

To sprawia, że RAG działa **inteligentnie i stabilnie**, zamiast halucynować lub szukać przypadkowych fragmentów.

Popularne

Technologie AI

Chmurowe, Open-Source i On Premise

Modele w chmurze

OpenAI / Anthropic / Google

- **OpenAI**
  - GPT-4.1 / GPT-4.1-mini
  - o1 / o1-mini (reasoning)
  - GPT-4o, GPT-4o-mini (multi-modal)
- **Anthropic**
  - Claude 3.5 Sonnet
  - Claude 3.5 Haiku
  - Claude 3 Opus
- **Google**
  - Gemini 2.0 Flash
  - Gemini 2.0 Pro
  - Gemini 1.5 Pro (długi kontekst)

Zalety

- najwyższa jakość modeli (reasoning, generowanie, kontekst, narzędzia)
- stabilne i przewidywalne API
- funkcje: structured outputs, tools, batch, embeddings wysokiej jakości

Wady / ograniczenia

- wysoki koszt przy dużej skali
- brak pełnej kontroli nad modelem
- ograniczenia regulacyjne: dane opuszczają organizację (zależnie od regionu)

Kiedy używać

- krytyczna jakość odpowiedzi
- zadania reasoning / coding / analiza dokumentów

Open-source

– Llama, Mistral, Qwen

- **Llama**
  - Llama 3.1 8B / 70B / 405B
  - Llama 3.2 3B / 11B (edge-first)
- **Mistral**
  - Mistral 7B
  - Mixtral 8x7B (MoE)
  - Nextral 8x22B
- **Qwen**
  - Qwen 2.5 (7B–72B)
  - Qwen2-Coder
  - Qwen-Plus (finetune'y)

Zalety

- pełna kontrola, możliwość hostowania on-prem / edge
- niskie koszty przy dużym wolumenie zapytań
- możliwość trenowania / fine-tune dla domeny

Wyzwania

- wymaga GPU i zespołu MLOps
- niższa jakość reasoning vs topowe LLM
- trudniejsze skalowanie i monitoring

Główne modele

- **Llama 3.x** – najlepszy ogólny jakościowo
- **Mistral / Mixtral** – świetna wydajność, MoE
- **Qwen 2** – bardzo dobre modele dla kodu

Quantization

– dlaczego ma znaczenie

Jak to działa?

- Standardowy model używa liczb **float32** (duża precyzja).
- Quantization zamienia je np. na **8-bit albo 4-bit**, czyli dużo mniejsze liczby.
- Model **zajmuje mniej pamięci**, a GPU/CPU **może go liczyć szybciej**.

Rodzaje

- 4-bit: GGUF, AWQ, GPTQ
- 8-bit: kompromis jakość/latency

Po co to?

- żeby model **zmieścił się na jednym GPU**, nawet na laptopie
- żeby **działał szybciej**
- żeby **koszt inferencji był dużo niższy**
- żeby **uruchomić duże modele open-source bez superkomputera**

Konsekwencje

- mniejsza precyzja modeli, czasem gorsze reasoning
- idealne do chatbotów domenowych i RAG

vLLM, Ollama

– jak uruchamiać modele lokalnie

vLLM

- najwyższa wydajność inference
- batching, continuous batching → wysokie throughput
- idealne pod API produkcyjne

Ollama

- idealne dla developerów lokalnie
- szybki test modeli open-source
- integracja z laptopem/desktopem

Vector tools

Przechowywanie i wyszukiwanie Embeddings

pgvector

- PostgreSQL + wektory
- idealne przy istniejącej infrastrukturze SQL
- wystarczające dla większości RAG

Pinecone

- wysokowydajne, globalne, skalowalne
- płacisz za jakość i replikację

Chroma

- open-source, szybkie POC
- proste API, ale trudniejsze w dużej skali

Cloud AI

– Azure, AWS, GCP

Azure OpenAI

- zgodność enterprise, regiony EU
- najlepsze pod regulowane branże
- drożej niż open-source, ale stabilnie

AWS Bedrock

- multi-model (Anthropic, Mistral, Amazon Titan)
- świetna integracja z AWS ekosystemem
- dobre dla dużych korporacji

GCP Vertex AI

- najlepsze narzędzia do pipelines i MLOps
- AutoML, feature store, model registry
- super dla firm ML-first

Kiedy co wybrać

- **Top jakość odpowiedzi** → OpenAI / Anthropic
- **Duża skala, optymalizacja kosztów** → open-source + vLLM
- **RAG + istniejąca infrastruktura SQL** → pgvector
- **Regulacje / polityki danych / sektor finansowy** → Azure OpenAI
- **Custom modele + pipelines ML** → GCP Vertex
- **Środowisko AWS** → Bedrock (integracje, IAM, VPC)

Inżynieria wymagań z AI

Problem, metryki i decyzje

Wymagania to decyzje

Nie tylko metryka. To wybór i zobowiązanie.

Znaczenie > forma.
— Czytelność dla biznesu po pierwsze.

- Kryteria decyzji: wartość, ryzyko, koszt, czas.
- Konsekwencje: techniczne, operacyjne, prawne.

- Testowalność. Każdy cel i metryka musi dać się zweryfikować.
- Świadoma rezygnacja. Wartość bywa w tym, czego nie budujemy.

Pytanie biznesowe

Od niejasnego do problemu DS

AI

- ujawnia niejednoznaczności.
- Wykrywa sprzeczności i luki.
- Proponuje doprecyzowania.
- Podpowiada pytania kontrolne.

Data Scientist

- Wybiera interpretację.
- Nadaje kontekst biznesowy.
- Ustala definicje pojęć (np. „churn”, „aktywny klient”).
- Zamyka temat decyzją i uzasadnieniem.

Metryka modelu vs metryka biznesowa

AI faworyzuje accuracy i demo.

- Łatwo pomija koszt błędów i ograniczenia.
- Ryzyko: wysoki wynik w notebooku, słaby efekt w biznesie.
- Pamiętaj o całości: koszt FN/FP, opóźnienia, stabilność.

Data Scientist

- Chroni cel biznesowy i koszt błędów.
- Definiuje metryki i progi (np. precision, MAE).
- Ustala ograniczenia: latency, fairness, prywatność.
- Wpisuje metryki do kryteriów akceptacji.

Ryzyko nad-modelowania

AI ma skłonność do „over-engineering”.

- Zbyt złożony model = trudniejsze utrzymanie i wyjaśnienie.
- Przedwczesne decyzje blokują eksperymenty.
- Tworzy się dług techniczny w pipeline.

Cel:

minimum złożoności, maksimum opcji.

- Parametryzuj cechy i progi, nie betonuj ich w kodzie.
- Dokumentuj warianty modeli i trade-offs.
- Odkładaj niekrytyczne decyzje na później.

Człowiek w pętli

Checkpointy – jakości na kamieniach milowych.

- Gating dla modeli przed wdrożeniem.
- Eskalacja przy niepewności wyników.
- Weryfikacja danych i dowodów.

Governance

Jakość jest w procesie - nie w narzędziach

- RACI: kto decyduje, kto doradza, kto zatwierdza.
- Audit trail: dane, wersja modelu, prompt, wynik.
- Rewizje i podpisy. Rozliczalność.
- „Stop” przy ryzyku dla klientów i zgodności (RODO, dyskryminacja).

Proces pracy z AI

Definiowanie problemu

Ustal osobne Procesy — (AI-pipelines)

Discovery:

- zbieraj źródła danych, definicje, ograniczenia.

Refinement:

- wykryj luki w danych, doprecyzuj cel i etykiety.

Validation:

- testuj na zbiorze odłożonym i przypadkach brzegowych.

Decision:

- uzasadnij wybór, ustal metryki i ryzyka.

Śledzenie i dowody

- Źródła: dane (tabele, zrzuty), systemy, eksperci domenowi.
- Wersja modelu i parametrów.
- Wersje cech i ustawienia treningu.
- Wyniki i porównania wariantów.
- Metryki NFR: latency, dostępność, stabilność.
- Dowody: notebooki, raporty z walidacji, linki.
- Diff zmian definicji celu i danych w czasie.
- Zgodność: polityki danych, regulacje, DPIA.

PROMPTY:

napraw opis problemu DS

Krok A

— Zidentyfikuj problemy

- Przejrzyj opis celu i dostępne dane.
- Wykryj niejednoznaczności i sprzeczności.
- Zidentyfikuj brakujące metryki sukcesu.
- Znajdź niejasne zmienne, etykiety i definicje.
- Wypisz cele bez ścieżki do decyzji biznesowej.

Krok B

— Ulepsz do dobrych praktyk

- Przepisz cel jako problem ML (klasyfikacja/regresja/prognoza).
- Dodaj metryki z progami i baseline.
- Doprecyzuj dane wejściowe, etykiety i walidacje.
- Rozwiąż sprzeczności explicite.
- Dodaj traceability do celów i źródeł danych.

... Krok P

– jak "Proces"

- Zapisz odkryte dobre/złe praktyki jako gotowy prompt na przyszłość

Artefakty i kryteria

– Stwórz proces: Prompty dla każdego kroku i kadego rodzaju artefaktu

Stwórz własną bibliotekę:

- Promptów (+ kontekst)
- Typów Artefaktów
- Szablonów
- Checklist i guardrails

- Brief (BRD/PRD): cel biznesowy, zakres, ryzyka, miara sukcesu.
- Backlog eksperymentów: hipotezy, iteracje, zadania.
- Kryteria akceptacji modelu: próg metryki, baseline.
- NFR: metryki, progi, metody pomiaru.
- Scenariusze testowe: reprezentatywne przypadki.
- Negatywne przykłady i edge-cases.
- Reguły danych: walidacje, źródła, retencja.
- Zależności i interfejsy: źródła danych, API predykcji, SLA.

Guardraile dla AI

- Definiuj zakres i format odpowiedzi.
- Zakaz halucynacji: wymagaj źródeł i wyników z kodu.
- Preferuj listy kontrolne i porównania.
- Pytaj o ryzyko i alternatywy.
- Zawsze kończ rekomendacją i uzasadnieniem.

Trenowanie

Asystenta AI

Zaufanie do AI

— analogia nowego pracownika

- Model AI jest jak **nowy pracownik**
- Ma:
  - wiedzę ogólną
  - doświadczenie zawodowe
- Nie ma:
  - znajomości naszej firmy i danych
  - kontekstu decyzji
  - wiedzy o ryzykach i priorytetach

Zaufanie **nie jest dane** — jest **budowane procesem**

Dlaczego na początku
nie ufamy odpowiedziom AI?

- Błędy nie wynikają ze „złej woli”
- Najczęstsze przyczyny:
  - brak kontekstu
  - brak zasad
  - brak informacji, czego **nie wolno**

Dokładnie jak u nowego pracownika

jeśli nie ma wytycznych to ...

„jakoś" to zrobi 😟

AI i nowy pracownik — to samo ryzyko

- Obaj:
  - chcą pomóc
  - potrafią analizować
  - mogą popełniać błędy
- Problem:
  - **bez instrukcji będą zgadywać**
- Rozwiązanie:
  - jasno powiedzieć **kiedy dopytać**
  - jasno określić **kiedy powiedzieć „nie wiem”**

Instrukcje = opis roli pracownika

Instrukcje mówią asystentowi:

- Kim jesteś (np. Data Scientist, nie decydent)
- Jaki masz cel (analiza, nie decyzja)
- Jak się zachować, gdy:
  - brakuje danych → **zadaj pytania**
  - są niejasności → **zaznacz ryzyko**
  - czegoś nie wiesz → **powiedz to wprost**
- Efekt:
  - mniej zgadywania
  - więcej odpowiedzialnych odpowiedzi

Zgadywać czy dopytać?

- Bez instrukcji:
  - AI „spróbuje pomóc” → **wymyśli**
- Z instrukcją:
  - AI **zatrzymuje się i pyta**
- Dokładnie jak w onboardingu:
  - brak zasad = improwizacja
  - jasne zasady = przewidywalność

Gotowe prompty = procedury pracy

- Prompty działają jak:
  - checklisty
  - procedury (SOP)
  - standardy analizy
- Dają:
  - spójny sposób myślenia
  - powtarzalne wyniki
  - mniejsze ryzyko błędów
- My i „pracownik”:
  - działamy według **tych samych zasad**

Guardrails = granice odpowiedzialności

Guardrails określają:

- ❌ czego **nie wolno**:
  - podejmować decyzji biznesowych
  - zakładać faktów o danych
- ⚠️ co jest ryzykowne:
  - prawo
  - compliance
  - dane osobowe (RODO)
- ✅ co jest OK:
  - analiza
  - porównanie modeli
  - wskazanie ryzyk
- Jak regulamin pracy w firmie

Jak naprawdę wygląda uczenie się

Skuteczny proces (człowiek i AI):

1. Pokazujesz przykład
2. Wyjaśniasz **dlaczego**
3. Prosisz o wyjaśnienie własnymi słowami
4. Sprawdzasz wynik (np. metryka na zbiorze testowym)
5. Tłumaczysz:
   - co jest dobre
   - co złe
   - dlaczego

- Cel: **rozumienie**, nie zgadywanie

Narzędzia = samodzielność

- Dostęp do:
  - dokumentacji
  - systemów
  - danych (tylko odczyt)
- Efekt:
  - pracownik działa samodzielnie
  - AI nie zgaduje, tylko sprawdza
- Bez narzędzi:
  - każdy musi pytać o wszystko

Podsumowanie

Chat lub pracownik:

- muszą poznać:
  - nasz styl pracy
  - nasze cele
- muszą nauczyć się:
  - terminów
  - procesów
  - granic
- na początku:
  - pełna kontrola
- później:
  - wspólne procedury
  - nauka na błędach
- Zaufanie:
  - nie do „inteligencji”
  - do **procesu, który kontrolujemy**

Antywzorce w pracy z AI

(i nowym pracownikiem)

- ❌ „AI powinno samo wiedzieć”
  - brak kontekstu = zgadywanie
- ❌ Brak jasnej roli
  - raz Data Scientist, raz decydent
- ❌ Brak granic
  - AI „pomaga” tam, gdzie nie wolno
- ❌ Kara za powiedzenie „nie wiem”
  - efekt: pewne, ale błędne odpowiedzi
- ❌ Brak informacji zwrotnej
  - te same błędy powtarzane w kółko
- ❌ Traktowanie jednego wyniku modelu jak prawdy
  - brak procesu = brak zaufania

Ćwiczenie — AI Onboarding

Cel:

- Doświadczyć, że **zaufanie do AI buduje się jak do pracownika**

Krok 1 – Nadanie roli:

- Opisz AI:
  - kim jest
  - czego **nie robi**
  - jaki ma cel

Ćwiczenie — AI Onboarding

Krok 2 – Zasady:

- Kiedy ma:
  - dopytać
  - powiedzieć „nie wiem”
  - zaznaczyć ryzyko

Krok 3 – Przykład:

- Pokaż 1 dobry notebook z analizą
- Wyjaśnij **dlaczego jest dobry**

Ćwiczenie — AI Onboarding

Krok 4 – Sprawdzenie:

- Poproś AI o podobne zadanie
- Oceń:
  - co zrobiło dobrze
  - co źle
  - dlaczego

Krok 5 – Refleksja:

- Co zmieniło się w jakości odpowiedzi?
- Co trzeba było doprecyzować?
- Jakie guardrails były kluczowe?

Ćwiczenie
— AI Onboarding

Krok 6 - Proces:

Zapisz to jako pliki:

- Instrukcje dla agenta
- Wzór promptów dla kroków
- Checklist - jakość odpowiedzi
- Przykładowe zadanie

Nie pozwól by wiedza "uleciała"

– Ty pamiętasz, nowy czat już nie.

Model Context Protocol (MCP)

– Wspólny język dla modeli i agentów

Model Context Protocol

— co to jest?

- **Otwarty standard** łączący aplikacje AI (klient/host) z narzędziami i danymi (serwery) przez wspólny protokół
- Metafora: **„USB-C dla AI”** — jeden interfejs, wiele integracji
- Cel: ograniczyć problem „N modeli × M integracji” → **serwery są wymienne i wielokrotnego użytku**

Źródła: modelcontextprotocol.io, Anthropic announcement

Do czego można użyć MCP

- Udostępnianie modelowi **narzędzi (tools)**: akcje do wykonania (np. zapytanie SQL do hurtowni w trybie read-only, uruchomienie notebooka, operacje na repo)
- Udostępnianie **zasobów (resources)**: dane do odczytu/odwołania (pliki CSV, API, dokumenty, wyniki zapytań)
- Udostępnianie **promptów (prompts)**: parametryzowane szablony poleceń / workflow
- Budowanie „copilota z rękami”: model **nie tylko pisze**, ale może **wykonać krok** i zwrócić wynik

Źródła: MCP docs, Build an MCP server

Praktyczne przykłady serwerów MCP

(popularne „klocki”)

- **Filesystem** — czytanie/zapisywanie plików (np. CSV, Parquet) w kontrolowanych katalogach
- **Git** — analiza repo, gałęzie, diffy, historie
- **Fetch** — pobieranie i czyszczenie treści stron pod LLM
- **Time** — operacje na czasie / strefach / datach
- **Memory** — pamięć długoterminowa / graf wiedzy

Gdzie szukać: Official MCP Registry, registry repo

Jakie „rodzaje” MCP?

(transport / komunikacja)

- MCP definiuje **kontrakt** (wiadomości, narzędzia, zasoby), a transport mówi **jak się połączyć**
- Najczęstsze transporty:
  - **stdio** — lokalny proces (stdin/stdout), idealne dla dev-tooling
  - **HTTP / streamable HTTP** — zdalne serwery (łatwiejszy hosting, audyt, auth)
  - **SSE** — wspierane jako „legacy” w części integracji

Źródła: Use MCP servers in VS Code

Jak dodać MCP do VS Code

- MCP w VS Code: instrukcje i konfiguracje w dokumentacji Copilot
- Włączenie galerii serwerów:
  - ustaw `chat.mcp.gallery.enabled`
  - w Extensions wpisz `@mcp` lub użyj komendy **MCP: Browse Servers**
- Instalacja serwera:
  - do profilu użytkownika (globalnie) albo do workspace (dla zespołu)

Źródło: Use MCP servers in VS Code

VS Code

konfiguracja serwera w `.vscode/mcp.json`

- Konfiguracja w repo pozwala wersjonować i dzielić serwery w zespole
- Przykład: lokalny serwer (stdio) + serwer zdalny (http)

```
{
  "servers": {
    "mini-mcp": {
      "command": "node",
      "args": ["./dist/index.js"]
    },
    "github-mcp": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp"
    }
  }
}
```

Źródła: VS Code MCP servers, GitHub Copilot MCP

Co jeszcze może MCP

(ponad „tool calls”)

- Ułatwia budowę agentowych workflow: **wielokrokowe zadania** z weryfikacją wyników
- Wspiera elementy typu:
  - **authentication** (dla serwerów zdalnych)
  - dodatkowe **instrukcje serwera**, ograniczenia „roots”
  - mechanizmy „elicitation/sampling” (zależnie od klienta)
- Praktycznie: standardowy „adapter” do narzędzi + możliwość narzucenia governance

Źródło: VS Code MCP developer guide

Minimalny MCP w TypeScript:

krok 1 — init + install

- Cel: najprostszy serwer MCP po **stdio** z jednym tool'em `hello_world`
- Instalujemy SDK i narzędzia do uruchamiania TS

```
mkdir mini-mcp && cd mini-mcp
npm init -y
npm i @modelcontextprotocol/sdk
npm i -D typescript tsx @types/node
npx tsc --init
```

Źródła: @modelcontextprotocol/sdk (npm), Build an MCP server

Minimalny MCP w TypeScript:

krok 2 — Server + connect (stdio)

```ts
// src/index.ts
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";

const server = new Server(
  { name: "mini-mcp", version: "0.1.0" },
  { capabilities: { tools: {} } }
);

const transport = new StdioServerTransport();
await server.connect(transport);
```

- `Server` opisuje możliwości (capabilities)
- `StdioServerTransport()` = komunikacja przez stdin/stdout (idealna lokalnie)

Źródło: @modelcontextprotocol/sdk (npm)

Minimalny MCP w TypeScript:

krok 3 — tool `hello_world`

```
// ...

server.registerTool(
  "hello_world",
  {
    title: "Hello World",
    description: "Zwraca powitanie dla podanego imienia",
    inputSchema: { name: z.string() },
  },
  async ({ name }) => {
    return {
      content: [{ type: "text", text: `Hello, ${name}!` }],
    };
  }
);
```

Serwer odpowie automatycznie na zapytania Chata:

- `ListTools…` = discovery (co potrafi serwer)
- `CallTool…` = wykonanie i zwrot wyniku do klienta

Uruchomienie serwera

- podpięcie w VS Code (mini-checklista)
- Dev run:

```
npx tsx src/index.ts
```

- Albo build do `dist/` i start node:

```
npx tsc
node dist/index.js
```

- Dodaj do `.vscode/mcp.json` (workspace) i zrestartuj sesję czatu

Źródło: Use MCP servers in VS Code

Docker dla Data Scientist AI

Jak uruchamiać narzędzia AI bez instalowania ich ręcznie

Docker dla Data Scientist AI

Jak uruchamiać narzędzia AI bez instalowania ich ręcznie

- Docker = **kontener na gotowe narzędzie**
- Bez instalacji Javy, Node, Pythona i bibliotek ML
- Jedna komenda → działa
- Idealny do:
  - serwerów AI
  - MCP
  - narzędzi analitycznych
  - PoC i warsztatów

Dlaczego Docker jest ważny dla Data Scientist?

- Data Scientist **nie walczy z konfiguracją** środowiska
- Data Scientist **uruchamia gotowe narzędzia**
- Docker daje:
  - powtarzalność
  - brak „u mnie działa”
  - szybki start PoC
- AI + Docker = **lokalne, kontrolowane środowisko**

Podstawowe pojęcia Dockera (intuicyjnie)

- **Obraz (image)** – „zestaw instalacyjny” aplikacji
- **Kontener (container)** – uruchomiona aplikacja
- **Port** – jak wchodzisz do aplikacji przez przeglądarkę
- **Volume** – dane, które nie znikają po restarcie

🧠 *obraz = przepis, kontener = gotowe danie*

Jak wygląda uruchamianie aplikacji w Dockerze?

- Bez Dockera:
  - instalacja
  - konfiguracja
  - konflikty wersji bibliotek
- Z Dockerem:
  - `docker run ...`
  - aplikacja działa od razu
- Dla Data Scientist:
  - **nie interesuje Cię jak**
  - **interesuje Cię że działa**

Porty – jak „wejść” do aplikacji

- Aplikacje w Dockerze są „zamknięte”
- Port = **drzwi do środka**
- Przykład:
  - aplikacja słucha na porcie `8080`
  - mapujesz ją na `localhost:8080`
- Efekt:
  - otwierasz przeglądarkę
  - widzisz UI (np. Jupyter) / API / MCP

Zmienne środowiskowe (ENV) – ustawienia bez kodu

- ENV = **konfiguracja aplikacji**
- Przykłady:
  - klucz API do LLM
  - tryb DEV / PROD
  - adres bazy danych (np. Postgres)
- Zaleta:
  - nie zmieniasz obrazu
  - tylko ustawienia

Docker Hub – sklep z gotowymi narzędziami

- Repozytorium gotowych obrazów
- Bazy danych (Postgres), Jupyter, Ollama, MCP
- Szukasz → uruchamiasz → testujesz

Docker vs Docker Compose

Docker

- Jedna aplikacja
- Jedna komenda

Docker Compose

- Zestaw aplikacji
- Jeden plik `docker-compose.yaml`
- Idealne do stosu: Jupyter + Postgres + MLflow

docker-compose.yaml – serce konfiguracji

- Jeden plik = cała konfiguracja
- Obrazy, porty, ENV, wolumeny
- Zero kodu

MCP + Docker

- MCP = serwer narzędzi dla AI
- Docker = szybkie i bezpieczne uruchamianie
- Idealne środowisko dla Data Scientist

Oficjalny katalog MCP w Dockerze

https://docs.docker.com/ai/mcp-catalog-and-toolkit/

- Gotowe serwery MCP
- Instrukcje Compose
- Punkt startowy bez kodowania

Mentalny model Data Scientist

- Docker = START
- Compose = scenariusz
- MCP = rozszerzenie AI
- AI = asystent
- Ty = decyzyjny użytkownik

AI – Koncepcje vs Narzędzia

(dla Data Scientistów)

Model vs Narzędzie (produkt)

MODEL (koncepcja)

- Matematyczny / statystyczny mechanizm generujący odpowiedzi
- „Silnik” rozumowania językowego
- Sam w sobie nie ma UI

NARZĘDZIE (produkt)

- Konkretna aplikacja używająca modelu
- Ma interfejs, ustawienia i integracje

Przykłady

- Model: LLM
- Narzędzie: GPT-5.2, Claude, Gemini

Perspektywa Data Scientist Model = kompetencja
Narzędzie = pracownik

Chat vs Model

CHAT

- Interfejs rozmowy
- Przechowuje kontekst rozmowy (czasowo)

MODEL

- Generuje odpowiedzi statystycznie
- Nie pamięta poza kontekstem

Przykłady

- ChatGPT
- Gemini Chat

Copilot vs Chat

COPILOT

- AI wbudowana w narzędzie pracy
- Dostęp do plików, backlogu, dokumentów

CHAT

- Brak kontekstu systemowego

Przykłady

- GitHub Copilot
- Microsoft Copilot

Asystent vs Agent

ASYSTENT

- Reaguje na polecenia
- Nie działa samodzielnie

AGENT

- Ma cel i kroki działania
- Może używać narzędzi

Przykłady

- Asystent: ChatGPT
- Agent: AutoGPT, MCP Agent

Prompt vs Instrukcja

PROMPT

- Jednorazowe polecenie

INSTRUKCJA

- Stałe zasady zachowania

Token vs Kontekst vs Okno kontekstu

TOKEN

- Jednostka tekstu

KONTEKST

- Wszystko co model widzi

OKNO KONTEKSTU

- Maksymalny rozmiar kontekstu

Projekt (Custom GPT) vs Chat

PROJEKT

- Zapisana konfiguracja
- Instrukcje, pliki, narzędzia

CHAT

- Rozmowa ad-hoc

Przykłady

- Custom GPT
- Gemini Gem

Notebook vs Dokument

NOTEBOOK

- Eksperyment krok po kroku (kod, dane, wykresy)

DOKUMENT

- Statyczny artefakt – raport z wynikami

Przykłady

- Copilot Notebook
- Jupyter Notebook

RAG vs Chat z plikami

RAG

- AI + wyszukiwanie w wiedzy

CHAT Z PLIKAMI

- Jednorazowy upload

Model ≠ Wiedza ≠ Prawda

- Model generuje odpowiedzi
- Wiedza pochodzi z danych
- Prawda wymaga walidacji (metryki, testy)

Mentalny model Data Scientist

- Model = silnik
- Chat = interfejs
- Copilot = pracownik
- Agent = wykonawca
- Prompt = pytanie
- Instrukcja = proces
- Projekt = rola
- RAG = pamięć organizacji

Prompt Engineering

jako element pipeline’ów ML z AI

Prompty

jako interfejsy

- Prompt = **API kontraktowe** między systemem a modelem

- **Wejście:**
  - formatowane dane,
  - kontekst,
  - rola,
  - ograniczenia

- **Wyjście:**
  - przewidywalna struktura
  - JSON, listy, schematy

Prompty są częścią **architektury systemu**, nie tylko dodatkiem

Hermetyzacja promptów:

- dedykowane moduły promptów
- spójne konwencje, nazewnictwo, parametry modeli

Prompty jako komponenty:

- „prompt-per-task” → separacja odpowiedzialności
- miks warstw: system → developer → user

Standardy, testowanie, wersjonowanie

Wersjonowanie promptów:

- repozytorium kodu
- PR review → porównanie jakości odpowiedzi
- tagowanie:

  `/prompts/v1/analysis/churn-summary.md`

Testowanie promptów:

- A/B testy jakości odpowiedzi
- regression tests: te same dane

  → stabilność wyjścia
- testy „adversarial”

  → sprawdzanie odporności na injection

Operacjonalizacja:

- SLO: czas odpowiedzi, spójność, topical precision
- monitoring driftu danych i efektu zmian promptów
- walidacja schematu odpowiedzi (JSON Schema)
- limity długości i precyzji odpowiedzi
- obserwacja tokenów → koszty i czas

Szablony

Prompt Templates

query:
„Podsumuj wyniki modelu churn i wskaż 3 główne czynniki odejść klientów.”

context:
„Dane zawierają informacje o klientach; wynik ma spełniać RODO i być audytowalny.”

→ model otrzyma spójny kontekst i dostarczy poprawny raport.

```
SYSTEM:
You are an expert assistant.
Provide validated structured output only.

USER QUERY:
{query}

CONTEXT:
{context}

INSTRUCTIONS:
1. Combine query + context.
2. No hallucinations.
3. If context is irrelevant, ignore it.
4. Missing info → return "missing_data".
```

Po co stosować

template prompt

Standaryzacja pracy

- Każdy prompt ma tę samą strukturę → mniej błędów, spójne rezultaty.
- Łatwiej tworzyć procesy, checklisty i workflow dla zespołu.

Powtarzalność i automatyzacja

- Jeden template może obsłużyć setki przypadków użycia.
- Można go wstrzykiwać do CI/CD, notebooków, raportów BI, Copilot, pipelines.

Lepsza kontrola jakości

- Wymuszenie formatów (JSON, tabela metryk, raport).
- Minimalizacja halucynacji dzięki kontraktowi wejście/wyjście.

Po co stosować

template prompt

Łatwiejsze testowanie i wersjonowanie

- Prompt jako artefakt → można go testować, porównywać i versionować w Git.
- A/B testy promptów stają się proste.

Większa odporność na błędne dane

- Template pozwala modelowi reagować na braki: zgłosić ryzyka, nie wymyślać.

Hermetyzacja logiki

- Template oddziela *logikę zadania* od *zmiennych danych* (query, context).
- Tak jak w programowaniu: parametr zamiast „wklejania kodu”.

Spójność komunikacji

- W zespołach DS, ML engineering, data engineering i business — każdy używa tych samych formatów, produktów wyjściowych i standardów.

OpenAI Chat API:

Structured Output

w kontekście API i integracji systemów

Structured output

model zwraca ściśle zdefiniowaną strukturę danych

Prompt - język naturalny:

```
Przeanalizuj ryzyka dla modelu scoringu kredytowego ...
```

Odpowiedź - struktura:

```
{
  "risks": ["data leakage"],
  "components": ["features: dochód, historia spłat"],
  "recommendations": ["add time-based split"]
}
```

Zazwyczaj w formacie JSON,
zgodnie z podanym schematem (np. JSON Schema).

To oznacza:

- Model nie generuje dowolnego tekstu,
  tylko **konkretny obiekt JSON**
- Każde pole ma z góry określony typ
  (string, number, array, enum...)
- Model nie może wyjść poza strukturę
  – nie wolno mu dopisać zdań, komentarzy, itp.
- Odpowiedź staje się
  walidowalna i deterministyczna

Structured output

Dlaczego jest kluczowe w integracji?

- Minimalizuje halucynacje dzięki twardemu schematowi odpowiedzi
- Umożliwia walidację na poziomie API (schema-first)
- Upraszcza mapowanie do kontraktów domenowych
- Stabilizuje integracje — **LLM zwraca dane, nie tekst**
- Pozwala budować *"deterministyczne"* pipeline'y (AI jako krok transformacji)

Model jako

Komponent API

LLM jako mikroserwis (model serving)
→ musi mieć kontrakt wejścia/wyjścia

- Structured output pełni rolę **API specification**
- Możliwość wdrożenia:
  - typowanych DTO (TS/Java/Kotlin/Python)
  - walidatorów (Pydantic/Zod/JSON Schema)
  - automatycznego generowania dokumentacji

```
{   // JSON Schema
    "response_format": {
        "type": "json_schema",
        "json_schema": {
        "name": "model_analysis",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
            "risks": {
                "type": "array",
                "items": { "type": "string" }
            },
            "components": {
                "type": "array",
                "items": { "type": "string" }
            },
            "recommendations": {
                "type": "array",
                "items": { "type": "string" }
            }
            },
            "required": [
                "risks",
                "components",
                "recommendations"
            ]
        }
```

Kluczowe parametry

ścisłego output Chat API

- `response_format:{type:"json_schema"}`
  - ścisły format odpowiedzi

messages - interfejs sterujący zachowaniem modelu

```
messages: [
    {
        "role": "system",
        "content": "Return JSON only."
    },
    {
        "role": "user",
        "content": "Analyze churn dataset description."
    }
]
```

- `max_output_tokens`
  - kontrola kosztu i bezpieczeństwa
- `strict:true` w schemacie
  - modeli **nie wolno** wyjść poza kontrakt
- `tool_choice:"required"`
  - forcer na zwrócenie struktury zamiast tekstu

Typowy

wzorzec architektoniczny

Pipeline danych → AI Layer → Validation → Domain → Integrations

1. Pipeline formuje twarde wymagania (schema)
2. Chat API generuje *draft* danych w strukturze JSON
3. Walidacja:
   - JSON Schema
   - Zod / Pydantic
4. Mapowanie do obiektów domenowych
5. Agregacje, raporty BI, systemy downstream, ...

Tools & Function Calling

W kontekście integracji systemów

Czym są

“Tools” i Function Calling?

Tools w OpenAI to sposób na wywoływanie zewnętrznych akcji przez model

- API
- bazy danych
- feature store
- kalkulatory
- parsery
- generatory wykresów
- agenci itp.

Model nie „zgaduje” formatu
— generuje **strukturalne wywołanie funkcji**.

Twoja aplikacja decyduje, co dalej zrobić.

Funkcje są kontraktem → JSON Schema określa:

- nazwę funkcji
- parametry
- typy danych
- strukturę komunikacji

Architektura integracji:

Model ↔ Tool

1. Klient wysyła zapytanie
→ user prompt (np. „podaj score churnu dla klienta 12345”).

2. Model ocenia kontekst
→ czy sama odpowiedź wystarczy, czy musi wywołać „tool”?

3. Model zwraca JSON “function_call”

```
{
  "name": "getChurnScore",
  "arguments": { "customer_id": "12345" }
}
```

4. Backend wykonuje funkcję
→ Twój kod, serwis modelu, DB, API zewnętrzne.

5. Wynik wraca do modelu jako `tool` message
→ Model generuje finalną odpowiedź dla użytkownika.

Perspektywa

MLOps

Kontrolowane, deterministyczne API

→ Model nie buduje zapytań tekstowych, tylko strukturalny JSON.

Idempotencja

→ narzędzia muszą być projektowane jak API
— przewidywalne.

Separacja odpowiedzialności

Model: rozumienie i decyzja *co* zrobić.
System: faktyczne wykonanie *jak i gdzie*.

Bezpieczeństwo

→ brak swobodnych komend, sandboxing na poziomie kontraktów.
→ walidacja JSON schema przed wykonaniem.

Perspektywa

MLOps

- **Orkiestracja workflow**
  (model podejmuje decyzje, a system wykonuje akcje)
- **Natural language → SQL / API**
  (zapytania biznesowe mapowane na funkcje)
- **Inteligentne asystenty DS**
  (tool do ładowania danych, walidacji schematu)
- **RAG + Tooling**
  (wyszukaj → policz → zwróć)
- **Autonomiczne czynności, ale kontrolowane**
  (np. przelicz score → zapisz w feature store → wyślij alert Slack)

Projektowanie funkcji:

zasady:

1. Jedna funkcja = jedna odpowiedzialność
Żadnych “doAll()”.

2. Płaskie schematy JSON
Model lepiej radzi sobie
ze strukturami 2–3 poziomów.

3. Deterministyczność
Funkcje muszą być przewidywalne i idempotentne.

4. Walidacja
Każdy argument przechodzi JSON Schema Validation.

5. Kontekst minimalny
Nie wkładaj logiki biznesowej do modelu
— w funkcji robisz core logic.

integracyjne

Best practices

Hermetyzacja promptów
→ Parametry, role, kontrakty kontrolowane po stronie serwera.

Observability
→ Loguj każde wywołanie toola wraz z promptem wejściowym.

Retry & kompensacje
→ Tool wyrzuca błąd
→ model może podjąć inną akcję.

Versioning
→ Nowe funkcje
= nowa wersja API + nowy system prompt.

Testy:

→ Testujesz funkcje niezależnie od modelu
→ Testy regresyjne promptów i schem JSON

Anti-patterns

- Funkcja „zrób wszystko”
- Zagnieżdżone i mega-skomplikowane JSONy
- Przekazywanie logiki do modelu zamiast do narzędzia
- Brak walidacji odpowiedzi modelu
- Tools w stylu: `exec_shell` (zaproszenie do exploitów)
- Tool → API → model → tool → infinite loop (bez kontroli)

Jak projektować warstwy architektury pod

Tools?

Warstwa 1 – Model Orchestrator
OpenAI Chat API z toolami.

Warstwa 2 – Tool Adapters
Jedna klasa na tool.
Odpowiedzialne za walidację inputu i mapowanie JSON → funkcja.

Warstwa 3 – Services / Domain Logic
Prawdziwa logika biznesowa poza modelem.

Warstwa 4 – Infrastructure
DB, HTTP, chmura, storage — poza wiedzą modelu.

Przykładowe

kategorie narzędzi

- **CRUD tools** → odczyt / zapis danych
- **Computation tools** → kalkulatory, silniki scoringu
- **Search tools** → vector search, SQL search
- **Integration tools** → Slack, e-mail, Power BI, GitHub
- **Automation tools** → workflow steps
- **Parsing tools** → ekstrakcja danych z PDF/CSV/logów

Jak połączyć to z RAG?

1. Model ocenia pytanie
2. Wywołuje `vectorSearch` tool
3. Z wynikami podejmuje decyzję o kolejnym narzędziu
4. Może ponownie odpalić kolejne tooly (multi-step)
5. Finalna odpowiedź tworzy się dopiero na końcu

Architektonicznie:
→ RAG staje się jednym z „tools” w pipeline
→ Nie mieszasz logiki biznesowej z promptem

Kiedy NIE używać Tools?

- Gdy odpowiedź modelu jest wystarczająca
- Gdy system jest prosty i ma mało integracji
- Gdy funkcja może być wykonana *taniej i szybciej* po stronie klienta

Tools są najważniejsze w systemach AI, które:

- mają side-effects
- integrują wiele API
- wymagają kontrolowanej logiki biznesowej

AI w projektowaniu systemów ML

— asystent Data Scientist i ML engineera

Myślenie o promptach jak o

narzędziu inżynierskim

- Prompty ≠ magia

  → **narzędzie pracy**, jak model card, ADR, RFC

- LLM traktujemy jak **komponent systemu**:
  ma własne ograniczenia i interfejs
- Każdy prompt powinien mieć:
  cel, zakres i kryteria jakości
- Data Scientist nie „pyta z ciekawości”,
  tylko świadomie **projektuje zapytanie**

Struktura promptów

w analizie danych i projektowaniu rozwiązań ML

Rola (system) - nadaje zakres, ton, odpowiedzialność:

- „Jesteś senior Data Scientistem…”

Kontekst

- istniejący pipeline i dane
- ograniczenia biznesowe, NFR-y, ryzyka
- założenia sektorowe (finanse, telco, medyczne)

Format odpowiedzi

- JSON, listy, schematy, tabelaryczne porównania

Zadanie analityczne

- „przeanalizuj ryzyka”,
  „zaproponuj warianty pipeline’u”
- „porównaj batch vs streaming w tym przypadku”

Kryteria jakości:

- prompting strukturalny (sekcje, checklisty)
- few-shot - przykłady
- chain-of-thought kontrolowany:
  - „pokaż tylko plan / wnioski”
  - „krótko”, „z przykładami”,
    „z wyjaśnieniem trade-offów”

Meta-prompts

- Prompty definiujące **jak** model ma myśleć, a nie **co** ma wygenerować
- Ustalają **proces**, styl, poziom szczegółowości, ograniczenia
- Używane do kontroli jakości i powtarzalności

Przykłady meta-promptów

- „Zachowuj się jak senior Data Scientist. Zanim odpowiesz — zapytaj o brakujące dane.”
- „Oceń jakość mojego opisu eksperymentu, znajdź ryzyka, wskazówki usprawnień.”
- „Przeanalizuj problem jak ekspert od feature engineeringu i zaproponuj zestaw cech z wariantami.”

Step-by-Step

Prompts

- Wymuszają **proces rozumowania** krok po kroku
- Minimalizują halucynacje i błędy logiczne
- Idealne przy: analizie danych, planowaniu eksperymentów, refaktoryzacji notebooków

Przykład „Rozwiąż ten problem w 5 krokach: analiza → opcje → decyzja → ryzyka → metryki do monitoringu.”

Struktura

- Krok 1: Zbierz wymagania
- Krok 2: Oceń kontekst i ograniczenia
- Krok 3: Porównaj opcje (algorytmy, cechy, architektury)
- Krok 4: Zaproponuj rekomendację
- Krok 5: Dodaj ryzyka i testy weryfikujące

Prompty

„Ask-Me-Questions”

Najważniejsze narzędzie Data Scientist: model **musi dopytać** o brakujące dane

- Redukuje błędne założenia
- Tworzy proces podobny do realnych warsztatów z biznesem

Przykład

- „Zanim zaproponujesz rozwiązanie, zadaj do 10 pytań, które są konieczne, aby uniknąć złych założeń.”

Typowe obszary pytań

- świeżość danych (SLA)
- wolumen danych i częstotliwość predykcji
- integracje zewnętrzne
- dane wrażliwe / compliance
- cykl życia danych
- krytyczność biznesowa
- ograniczenia budżetowe

Przykład

- „Zanim wygenerujesz model, zapytaj o definicję targetu, dostępne zdarzenia i okno predykcji.”
- „Najpierw sprawdź: czy kontekst jest kompletny? Jeśli nie — zapytaj.”

Meta-prompty

Jak wykorzystać AI jako eksperta od Prompt Engineeringu

Rola AI jako „prompt co-pilota”

- AI pełni rolę **eksperta-asystenta**, który pomaga:
  - doprecyzować cele,
  - strukturyzować prompty,
  - eliminować niejednoznaczności,
  - ustalić formaty wyjścia,
  - wykonać sanity-check promptu przed użyciem.

Instrukcja:

- „Najpierw oceń mój prompt → listuj problemy.”
- „Potem zaproponuj ulepszoną wersję.”
- „Na końcu wygeneruj wersję produkcyjną zgodną z moimi ograniczeniami.”

Meta-prompt

„Tworzenie promptów wielorundowych”

AI jako architekt promptów:

- projektuje **scenariusz dialogu**,
- definiuje kolejne kroki,
- dodaje instrukcje typu:
  - „Zawsze zadawaj pytania, jeśli czegoś brakuje”
  - „Nie wykonuj zadania, dopóki nie potwierdzę specyfikacji”
  - „Użyj chain-of-thought, ale nie pokazuj go w output”

Prompty jako

Narzędzie Data Scientist

Wspomaganie

projektowania systemów ML

AI jako narzędzie **augmentujące**,
a nie zastępujące Data Scientista

- przyspiesza analizę,
  nie zastępuje decyzji
- świetna w eksploracji opcji,
  słabsza w ocenie trade-offów biznesowych
- warto traktować model jako
  sparring partnera dla Data Scientista

Generowanie propozycji

wzorców ML

AI jako „katalog wzorców na żądanie”

- dobór wzorca do kontekstu domenowego
  - batch vs streaming,
  - feature store,
  - model serving,
  - champion / challenger
- omówienie plusów/minusów i typowych pułapek
- generowanie wariantów rozwiązania pod konkretne ograniczenia
  - latency,
  - throughput,
  - RODO/PII,
  - koszt inferencji,
  - zero-downtime
- analiza dopasowania:
  - czy wzorzec jest adekwatny do skali i ryzyka?
  - typowe antywzorce:
    - „deep learning for everything”,
    - „streaming bez potrzeby”

Analiza

zależności i ryzyk w danych i modelach

LLM jako narzędzie do *risk-spotting*

analiza zależności między źródłami danych i cechami na podstawie kodu / opisów

- wykrywanie cykli, niejawnych zależności (np. data leakage)
- szacowanie skutków zmian (wpływ na downstream)

identyfikacja miejsc podatnych na awarie:

- single point of failure
- brak timeoutów / retry / fallbacku modelu

ocena gotowości na skalowanie:

- wąskie gardła I/O
- ograniczenia GPU/LLM, cache miss rate, cold starts modelu

analiza zgodności z wymaganiami niefunkcjonalnymi

- SLO, świeżość danych, polityki danych, compliance

Dokumentowanie projektów

AI jako szybki generator dokumentacji,
a nie „źródło prawdy”

tworzenie z AI:

- ADR (Architecture Decision Records)
- model cards i RFC / technical proposals
- standardów danych (data contracts)

automatyczne streszczenia
dokumentów projektowych

porządkowanie istniejącej dokumentacji:

- deduplikacja, scalanie, porównywanie wersji
- generowanie checklist walidacji modelu

Generowanie diagramów pipeline’ów

Współpraca AI ↔ narzędzia diagramowe

generowanie szkiców diagramów z opisu tekstowego

- diagram przepływu danych (źródło → ETL → model → serving)
- sequence diagrams
- flowcharts

aktualizacja diagramów na podstawie zmian w projekcie

- AI → tekst → automatyczna transformacja → diagram

utrzymywanie spójności między kodem, dokumentacją i diagramami

Integracja AI w procesach projektowych

Jak realnie używać AI w projektach DS?

AI jako „reviewer” decyzji modelowych

- ocena konsekwencji wyborów
- wskazanie brakujących założeń

AI jako wsparcie w warsztatach z biznesem

- generowanie pytań i scenariuszy „what if”

AI w integracji systemów

- generowanie kontraktów API (OpenAPI/AsyncAPI)
- transformacje danych
  (mappings, validation)
- testy integracyjne i scenariusze E2E

AI w integracji danych

- propozycje schematów, rozwiązywanie konfliktów
- generowanie reguł ETL/ELT,
  walidacja jakości danych

Ograniczenia i pułapki

– które Data Scientist MUSI znać

- halucynacje → konieczna walidacja techniczna
- brak pełnej wiedzy o kontekście organizacji
- modele nie rozumieją kosztów operacyjnych i polityk bezpieczeństwa
- potrzeba audytowalności:
  - trace logi promptów
  - wersjonowanie promptów i artefaktów

ryzyka:

- prompt injection w dokumentacji
- błędne wzorce wynikające z niepełnych danych
- przeszacowanie możliwości modeli

Best practices dla Data Scientistów

Jak pracować efektywnie z AI?

- używaj **structured prompting**:
  role → kontekst → cel → format
- utrzymuj **źródło prawdy** poza modelem (repo, rejestr modeli, data dictionary)
- generuj wiele wariantów i porównuj
- twórz szablony promptów dla:
  - analizy ryzyk
  - decyzji modelowych
  - generowania diagramów pipeline'u
- stosuj podejście: „AI jako junior Data Scientist + senior review"

Podsumowanie

AI sam nie projektuje rozwiązania
— pomaga je zaprojektować LEPIEJ

- szybciej eksploruje opcje
- wykrywa zależności i ryzyka
- przyspiesza dokumentowanie
- automatyzuje powtarzalne elementy procesu

- ...
- pozostawia decyzje strategiczne **Data Scientistowi**

AI w procesie data science

hype a rzeczywistość

Vibe coding

"Kodowanie intuicyjne"

W praktyce vibe coding przyjmuje dwie formy:

Czysty vibe coding

– pełne zaufanie do wyników AI.

Sprawdza się przy szybkich eksperymentach lub „weekendowych projektach do wyrzucenia”, gdzie liczy się tempo, a nie perfekcja.

Odpowiedzialne programowanie z asystą AI

– AI działa jak inteligentny współprogramista.

Ty prowadzisz, weryfikujesz, testujesz i rozumiesz wygenerowany kod. To Ty zachowujesz kontrolę i pełną odpowiedzialność za efekt końcowy.

Vibe coding

"Kodowanie intuicyjne"

Najpopularniejsze narzędzia do „ai - codingu”:

- **GitHub Copilot** – asystent AI w IDE, podpowiada kontekstowo kod. Obsługa kontekstu, MCP, itd.
- **Cursor** – (fork VS Code) z generowaniem i debugowaniem kodu.
- **WindSurf** – edytor AI do automatyzacji DevOps i pracy w chmurze.
- **Replit** – chmurowe IDE z funkcją AI „Get Unstuck”.
- **Cody (Sourcegraph)** – asystent AI z dostępem do repozytorium.
- **Bolt (StackBlitz)** – generuje aplikacje full-stack.
- **v0 (Vercel)** – generuje UI (React + Tailwind)
- **ChatGPT** – uniwersalny asystent AI do generowania i debugowania kodu.

Vibe Coding

– skrócona checklista dobrych praktyk

- Najpierw Wizja i projekt
  - Wiedz, co i po co badasz.
  - Myśl z perspektywy biznesu i pytania badawczego.
  - Najpierw cel, metryka sukcesu i opis danych.

- Środowisko i stack
  - Używaj popularnych, dobrze wspieranych technologii (AI zna je najlepiej)
  - Folder /instructions z przykładami i zasadami
  - Czyste repozytorium GIT i często commity .

- Praca z AI
  - Pisz konkretne, jednoznaczne prompty.
  - Dziel duże notebooki i funkcje na małe kroki.
  - Restartuj chat, gdy AI zaczyna się gubić
  - Podawaj tylko istotne pliki i kolumny, nie wklejaj surowych danych (PII), odwołuj się do przykładów.
  - Audytuj kod, waliduj dane, ukrywaj sekrety i klucze API, weryfikuj uprawnienia i błędy

- Mindset
  - Jasność i struktura ponad szybkość.
  - Krótkie pętle, szybka weryfikacja.
  - Buduj, testuj, poprawiaj — vibe z dyscypliną.

Kilka praktycznych wskazówek

- Najpierw opowiedz mi swój plan — nie pisz kodu.
- Podaj kilka opcji, zaczynając od najprostszej — bez kodowania.
- Pomyśl tyle, ile trzeba, i zadawaj pytania, jeśli potrzebujesz więcej informacji.
- Wyszukaj powtórzenia lub zbędny kod i wypisz je.
- Skup się na zrozumieniu, jak działa kod, a nie na nauce jego pisania.

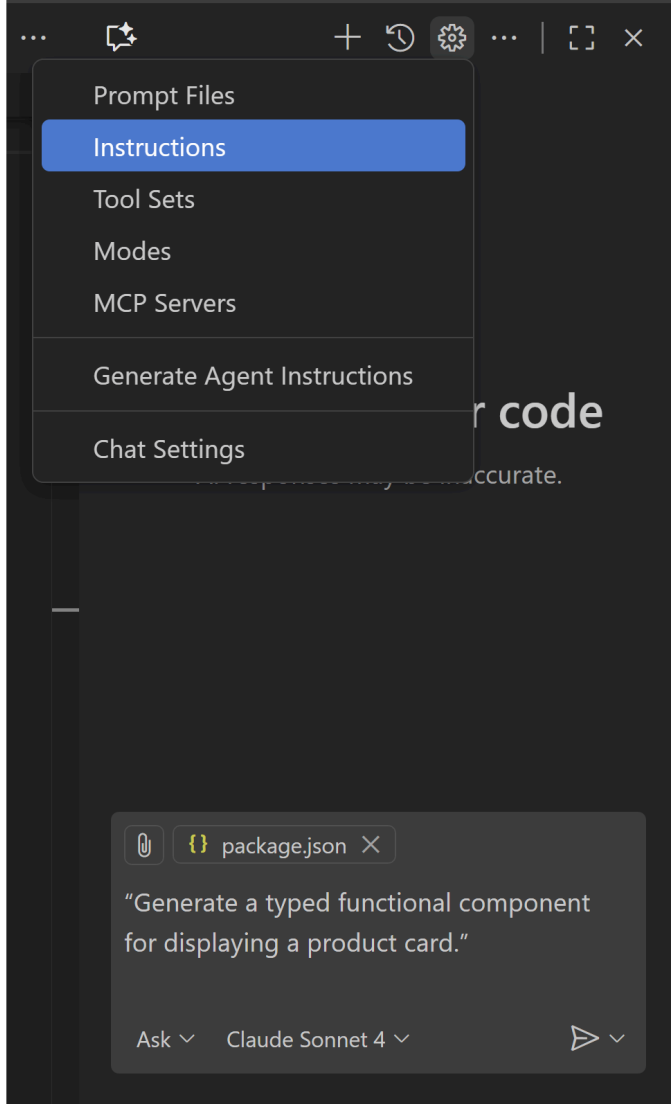
Copilot Chat

Okno Copilot Chat (panel boczny lub inline)

- /explain — wyjaśnij zaznaczony kod
- /fix — popraw błędy lub linter issues
- /tests — wygeneruj testy dla funkcji przetwarzającej dane (pytest)
- /docs — stwórz dokumentację lub komentarz
- /commit — zaproponuj opis do commita

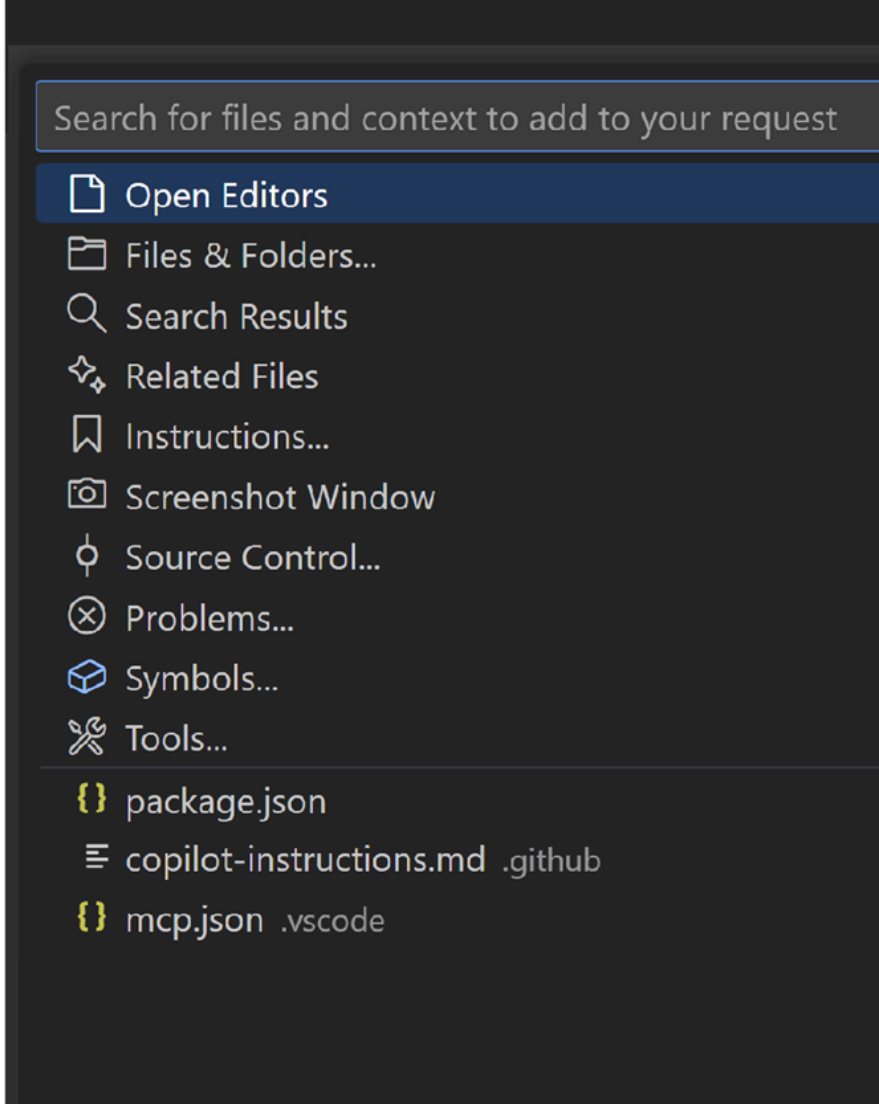
Inline Chat (Ctrl+I) - Chat kontekstowy otwierany w edytorze obok zaznaczonego fragmentu kodu.

- wyjaśnienie zapytania SQL lub transformacji pandas
- refaktoryzację komórki notebooka do funkcji
- konwersję pętli do operacji wektorowych w pandas
- dodanie testów lub komentarzy



Konteksty Copilot Chat

- Open Editors – aktualnie otwarte pliki.
- Files & Folders – wybrane pliki lub katalogi.
- Search Results – wyniki wyszukiwania.
- Related Files – powiązane testy, style, typy.
- Instructions – plik zasad i stylu (copilot-instructions.md).
- Screenshot Window – zrzut ekranu edytora.
- Source Control – zmiany z Git (diff, commity).
- Problems – błędy i ostrzeżenia z panelu Problems.
- Symbols – funkcje, klasy i moduły z kodu.
- Tools – integracje z narzędziami (pip, pytest, git).





Process AI-driven

draft → refine → verify → integrate

Wprowadzenie

do programowania z AI

Czym jest proces AI-driven

- Systematyczny sposób pracy z Copilotem
- Oparty na cyklu iteracyjnym,
  nie pojedynczym promptcie
- Łączy kreatywność AI z kontrolą data scientista

Dlaczego potrzebny jest proces

- Pojedynczy prompt = losowy wynik
- Proces = przewidywalne rezultaty i wyższa jakość
- Każdy etap ma inne pytania i metody

Rola Copilota

- **Partner**, nie generator
- Pomaga tworzyć, tłumaczyć, testować i dokumentować
- Wymaga kontekstu i kierunku, nie zgadywania

Draft

Uzyskaj pierwszą wersję kodu lub struktury, szybko i bez oceniania

Prompty:

- “Create base function that loads a CSV with sample data and prints basic info”
- “Generate folder structure for a X project using A + B
- “Bootstrap notebook with sections: data loading, EDA, model, conclusions using Z”

Kroki:

- Poproś Copilota o wygenerowanie pojedynczej funkcji
- Sprawdź, co stworzył, bez poprawiania
- Zapisz wersję jako `draft`

Refine

Uporządkuj, doprecyzuj i popraw jakość kodu

Prompty:

- “Refactor into smaller functions and add type hints”
- “Improve readability and follow our style rules”
- “Add type hints and docstrings to this function”
- “Extract data loading logic into a separate module”

Kroki:

Ulepsz funkcję, dodając type hints i obsługę błędów danych

Verify

Sprawdź poprawność, stabilność i jakość

Prompty:

- “Generate pytest tests for this function on a small data sample”
- “Check this file for performance issues and data leakage”
- “List potential runtime errors and fix them”

Kroki:

- Wygeneruj test dla funkcji
- Uruchom pytest
- Popraw błędy wskazane przez Copilot Chat

Integrate

Połącz dopracowane elementy w spójny system

Prompty:

- “Summarize this diff as a commit message”
- “Document public API for this module”
- “Integrate functions into a complete pipeline”
- “Explain how data flows through this pipeline”

Kroki:

- Zintegruj moduł ładowania danych i model w jeden pipeline
- Wygeneruj opis commita przez Copilota
- Sprawdź, czy notebook działa od początku do końca (Restart & Run All)

Copilot w Pull Requestach

Code-review na "naszych zasadach"

Zasada ogólna

Copilot “czyta” tylko to, co ma w kontekście:

- pliki repozytorium
- diff PR
- opis PR
- komentarze w PR

➡️ Twoje zasady muszą:

- **istnieć w repo**
- **pojawiać się w kontekście PR.**

Zasady w repo – źródło prawdy

Umieść wytyczne jako krótkie pliki:

- `docs/modeling`
  - `modeling-guidelines.md`
  - `feature-definitions.md`
- `docs/coding-style`
  - `python-style-guide.md`
- `docs/testing`
  - `testing-strategy.md`
- `docs/security`
  - `data-privacy-guide.md`

Dlaczego działa?
Copilot widzi pliki repo i potrafi je wykorzystać przy generowaniu zmian.

Włącz zasady do szablonu PR

`PULL_REQUEST_TEMPLATE.md` jako mini-prompt:

```
## Cel zmian

<!-- Krótko opisz problem i rozwiązanie. -->

## Kontekst analityczny

- [ ] Zgodne z definicjami cech i metryk (`docs/modeling/feature-definitions.md`)
- [ ] Zgodne z zasadami modelowania (`docs/modeling/modeling-guidelines.md`)
- [ ] Brak wycieku danych (data leakage) między zbiorem treningowym a testowym

## Checklist

- [ ] Zastosowano konwencje nazewnicze z `docs/coding-style/python-style-guide.md`
- [ ] Dodano testy zgodnie z `docs/testing/testing-strategy.md`
- [ ] Zwrócono uwagę na dane wrażliwe (`docs/security/data-privacy-guide.md`)
```

Komentarze w PR jako

“mini system prompts”

Używaj komentarzy typu:

- „Zrefaktoryzuj zgodnie z `feature-definitions.md`”
- „Proponuj zmiany zgodne z `python-style-guide.md`”
- „Zastosuj nasze zasady modelowania (patrz `modeling-guidelines.md`)”

Copilot doładuje te pliki do kontekstu.

Połącz Copilota z twardymi regułami

Copilot to asystent, nie policjant.

Egzekwuj zasady narzędziowo:

- ruff / black / mypy
- testy schematów danych (np. pandera, Great Expectations)
- testy na danych i metrykach (pytest)
- skanery bezpieczeństwa

Copilot pomaga „naprawić”,
ale to CI wymusza zgodność.

Narzędzia do statycznej analizy kodu są:

- przewidywalne
- konfigurowalne
- wytłumaczalne
- ścisłe

- ...
- AI nie jest.

Copilot Enterprise + baza wiedzy

Jeśli organizacja używa Copilot Enterprise:

- dodaj tam ADR-y, opisy metryk, playbooki analityczne
- Copilot w PR będzie mógł się do nich odwołać
- zwiększa trafność refaktoryzacji kodu analitycznego

Standaryzowane prompty dla reviewerów

Przykłady komentarzy:

- „Przeanalizuj PR pod kątem data leakage i definicji cech (patrz `modeling/feature-definitions.md`)."
- „Sprawdź zgodność ze style guide + zaproponuj brakujące testy."
- „Zidentyfikuj ryzyko ujawnienia danych wrażliwych (PII) lub sekretów."

Stosowane konsekwentnie → Copilot zaczyna trafiać częściej.

CODEOWNERS

AI review gate

Wzmocnij proces PR:

- `CODEOWNERS` dla kluczowych katalogów
- CI z linterami, testami danych, security scan
- Copilot wspiera autora i lead data scientista w przejściu review

CODEOWNERS

```
# This is a comment.

*       @username1

/docs   @username2

*.py    @dev-team
```

Gwiazdka `*` reprezentuje wszystkie pliki w repozytorium, a `@username1` jest przypisany jako właściciel kodu.

`/docs` określa, że `@username2` będzie właścicielem kodu wszystkich plików w katalogu docs.

`*.py` wskazuje, że wszelkie pliki Python w repozytorium należą do `@dev-team`.

Plan wdrożenia

w organizacji:

- Stwórz krótkie zasady w `docs/`
- Dodaj szablon PR z checklistami
- Ustaw linters + testy na danych
- Przygotuj zestaw komentarzy-promptów
- Przeszkol zespół (1h)
- (Opcjonalnie) Połącz z Copilot Enterprise

Efekt

Copilot zaczyna proponować zmiany:

- zgodne z metodyką analityczną
- spójne ze stylem organizacji
- zgodne z zasadami modelowania, testów i bezpieczeństwa danych

➡️ PR stają się szybsze, czytelniejsze i bardziej spójne.